King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 7
Trees

Terms 261

Reading Assignment

- "Data Structures and Algorithms in Java", 3rd Edition, Adam Drozdek, Cengage Learning, ISBN 978-9814239233
  - Chapter 6 Sections 1-6.
  - Section 6.4.3 regarding "Stackless Depth-First Traversal" and "Threaded Trees" is omitted.

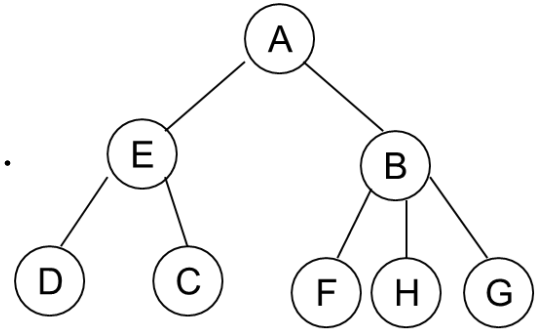
Objectives

- Discuss the following topics:
  - Trees, Binary Trees, and Binary Search Trees
  - Binary Trees implementations
  - Tree Traversal
  - Binary Search Tree
    - Binary Search Tree Searching
    - Binary Search Tree Insertion
    - Binary Search Tree Deletion
    - Why Binary Search Trees
    - Disadvantages of Binary Search Trees

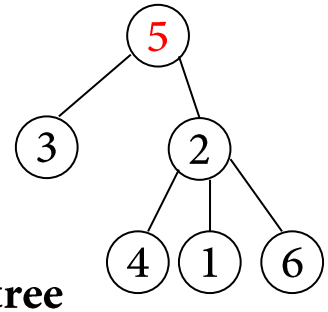
Definition of a Tree

A set of nodes storing elements such that the nodes have a parent-child relationship

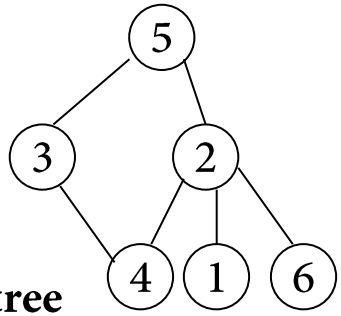
- A **tree** T is a non-linear data structure used to represent hierarchical (parent-child) relationships between elements.
- **Key characteristics:**
  - A tree consists of **nodes** (elements) and **edges** (relationship).
  - If T is nonempty, one node is designated as the **root**.
  - Every node except the root has exactly one parent.
  - A node can have zero or more children.
  - A tree contains no cycles. i.e. There is a unique path from the root to each node.



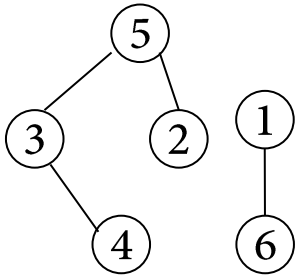
- nodes = {A,E,B,D,C,F,H,G}
- edges={(A,E),(A,B), (E,D),(E,C),(B,F),(B,H),(B,G)}



tree



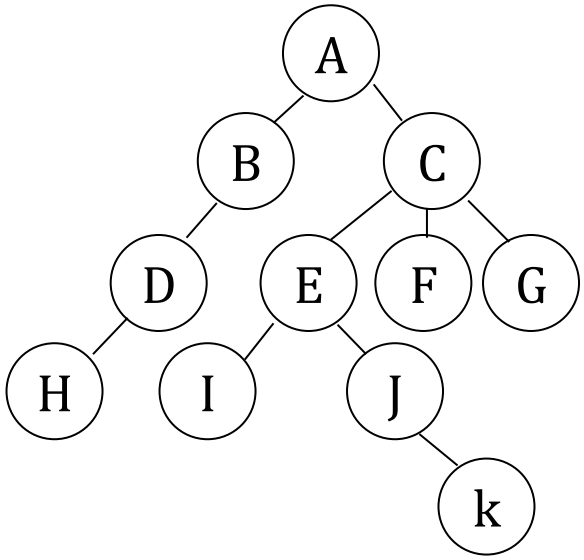
Not a tree



Not a tree

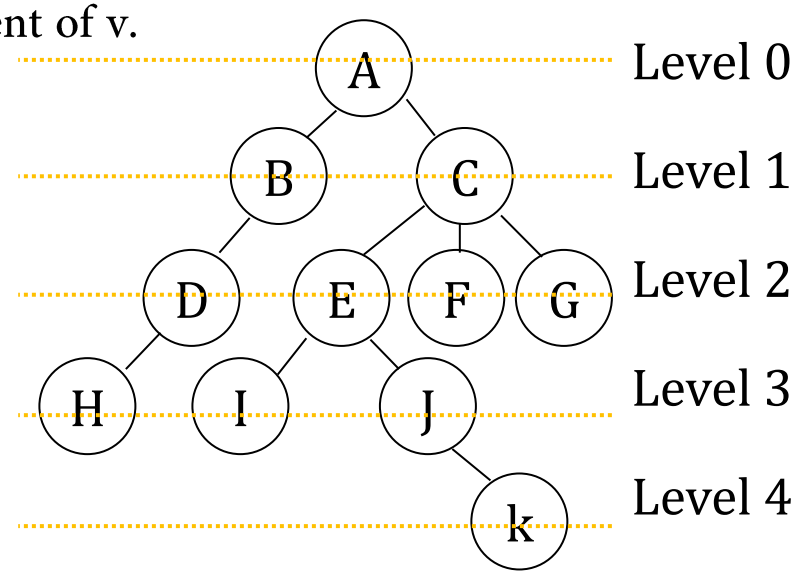
Tree Terminology

- **Root**: Top node of the tree
  - Example: A
- **Parent**: A node directly above another node ⟶
  - Example: B is parent of D
- **Child**: Node directly below another node
  - Example: D is child of B
- **Siblings**: Nodes with the same parent
  - Example: D and E
- **Degree of a Node**: The number of children of the node.
  - Example: C ⟶ degree 3
- **Leaf**: Node with no children
  - Example: H, I, F, G, K
- **Internal node**: Node with at least one child
  - Example: A, B, C, J
- **Subtree**: A node together with all its descendants
  - Example: Subtree rooted at B ⟶ B, D, E, G



Tree Terminology

- **Edge:** A pair of nodes (u, v) such that u is the parent of v.
  - Example: (A,B).
- **Path:** A sequence of nodes connected by edges.
  - Example: Path from A to D: **A ⟶ B ⟶ D**
- **Path length:** The number of **edges** in the path.
  - Example: the length of the path **A ⟶ B ⟶ D is 2**

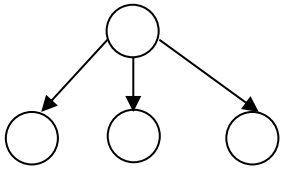


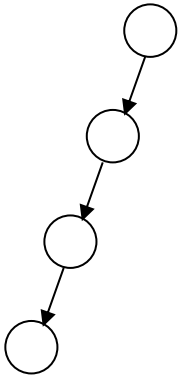
- Level (or depth) of a node v: The number of edges from the root to node v.
  - Example: B, C ⟶ Level 1
- Height of a node v: The number of edges on the longest path from v to a leaf.
  - Example: Height of C ⟶ **3**
  - The height of a nonempty tree is the height of its root node.
  - The height of an empty tree is -1.
- Ordered tree: A tree in which the children of each node are linearly ordered (usually from left to right). We can identify the children of a node as being the first, second, and so on

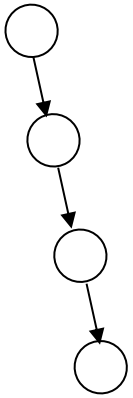
Example Trees

(a) is an empty tree

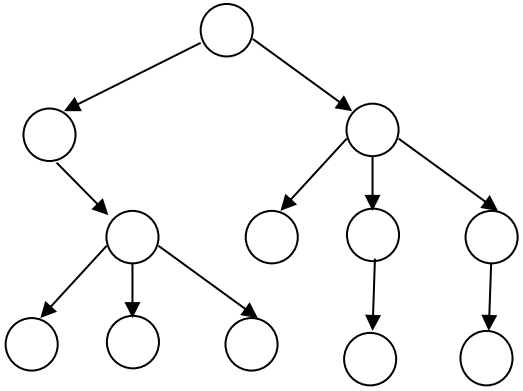


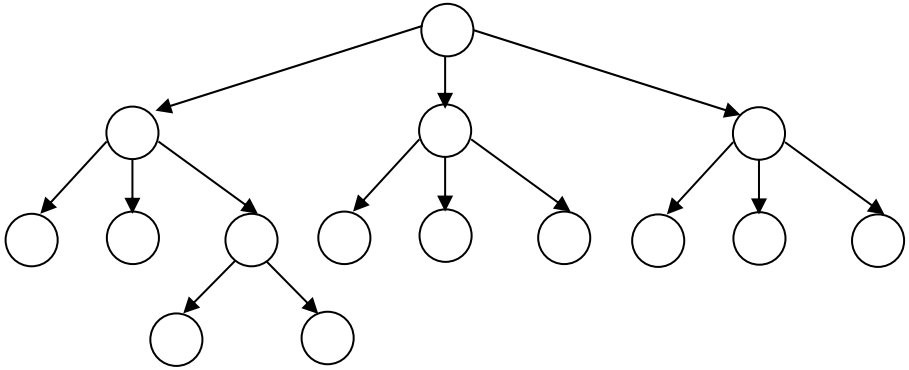






(a) (b) (c) (d) (e)





(f) (g)

Importance of Trees

- Trees are very important data structures in computing.
- They are suitable for:
  - Hierarchical structure representation, e.g.,
    - File directory.
    - Organizational structure of an institution.
    - Class inheritance tree.
  - Problem representation, e.g.,
    - Expression tree.
    - Decision tree.
  - Efficient algorithmic solutions, e.g.,
    - Search trees.
    - Efficient priority queues via heaps.

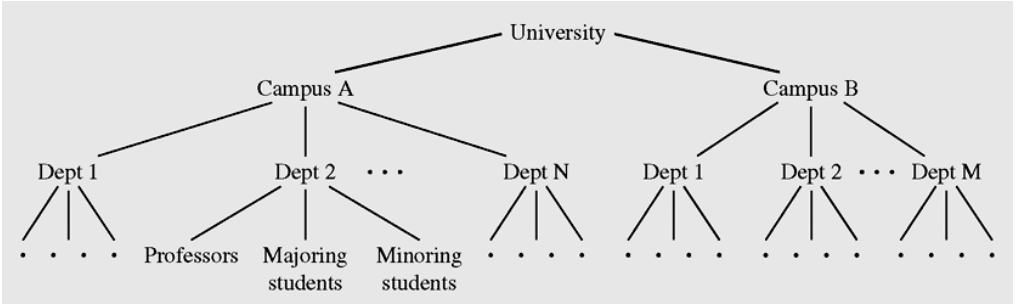


Figure 6-2 Hierarchical structure of a university shown as a tree

General Trees and their Representations (Contd.)

- In a general tree, each node can have any number of children.
  - To represent this in memory efficiently; we use a linked list-based structure.
- (1) Using a Linked List (or Array) of Children Pointers

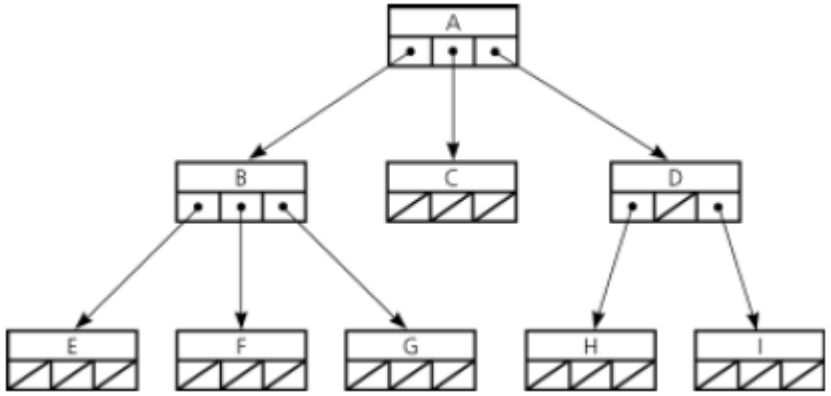
```
class Node<T> {
    private T data;
    private int degree;
    private Node<T>[degree] children;

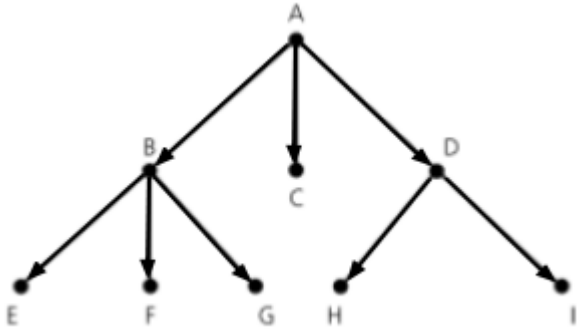
    public Node(T data) {
        this.data = data;
        children = new Node<T>[degree);
    }
    // . . .
}

public class GeneralTree<T> {
    private Node<T> root;

    public GeneralTree(){  }

    // . . .
}
```

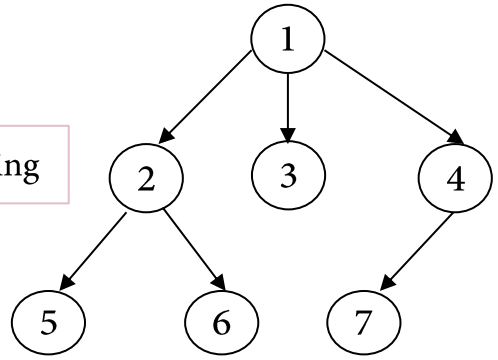




General Trees and their Representations

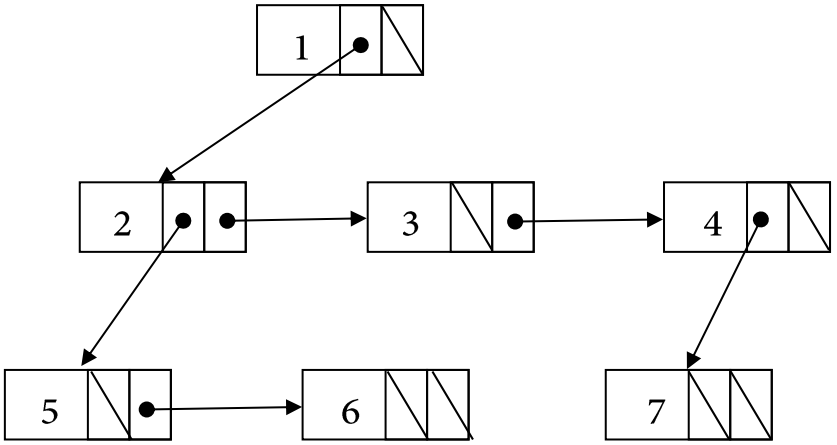
- (2) Left-Child Right-Sibling Representation
  - Each node contains:
    - Data (the value stored in the node)
    - A pointer to its first (leftmost) child
    - A pointer to its next sibling (the next child of the same parent)

Data	firstChild	nextSibling




```
class Node<T>{
  Node firstChild, nextSibling;
  T data;
  public Node(T data){
     firstChild = nextSibling = null;
     this.data = data;
   }
 }
 public class GeneralTree<T>{
   private Node root;

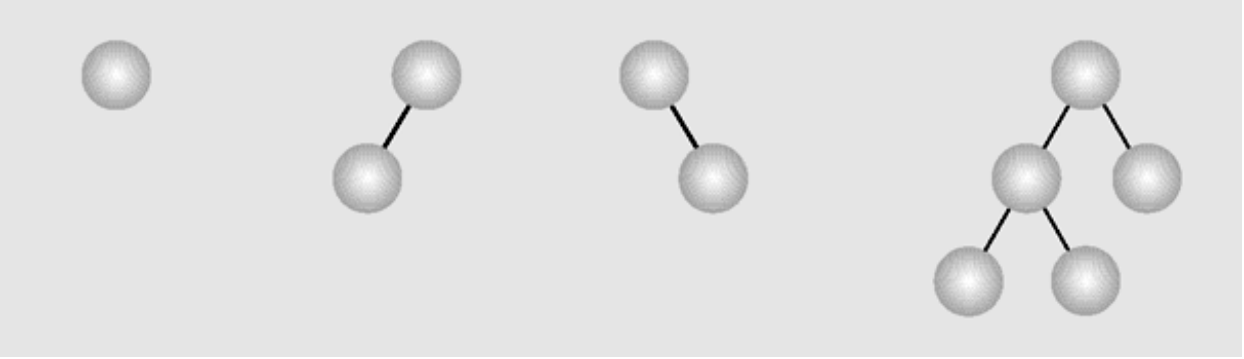
   public GeneralTree(){
      root = null;
   }
}
```



- The SECOND representation uses less memory; however, it may require more node access time.

Binary Trees

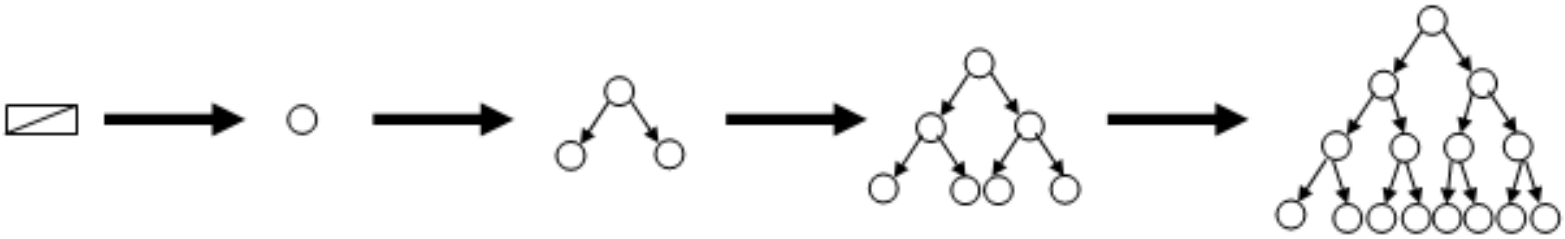
- A binary tree is an ordered tree tree in which each node has at most two children.
  - The two children are called:
    - **Left child**
    - **Right child**
  - The binary tree could be empty



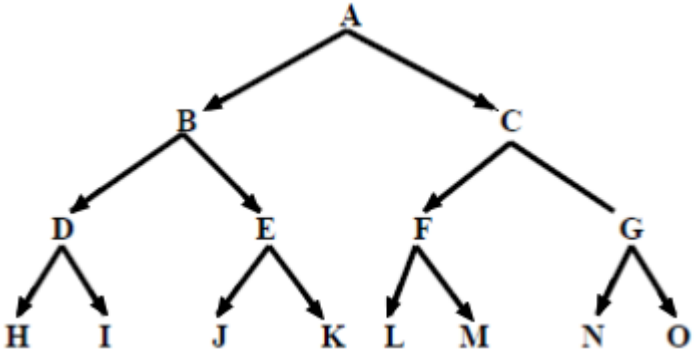
Examples of binary trees

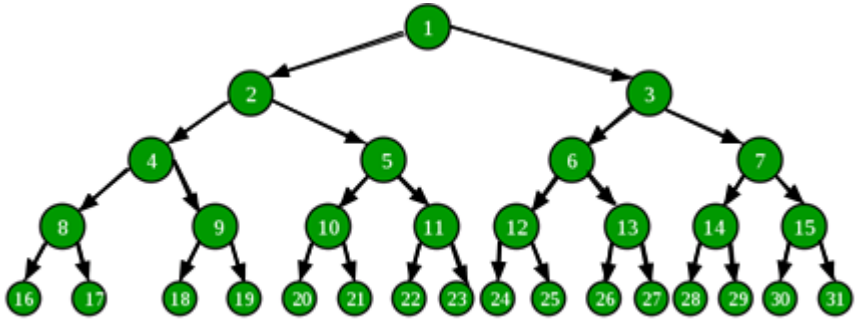
Types of Binary Trees

- A perfect binary tree is a binary tree in which each level k, $k \geq 0$, has 2^k nodes.



- Examples of perfect binary trees:

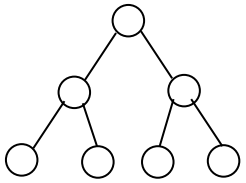




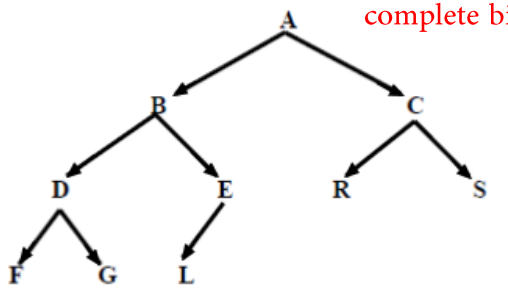
Types of Binary Trees (Contd.)

- A complete binary tree is a binary tree in which:
  - Each level k , k >= 0, except possibly the last level, contains the maximum number of nodes for that level, that is 2^k.
  - The last level may or may not contain the maximum number of nodes.
  - The nodes in the last level are filled from left to right with no gap.
- Thus, every perfect binary tree is a complete binary tree, but the opposite is not true.

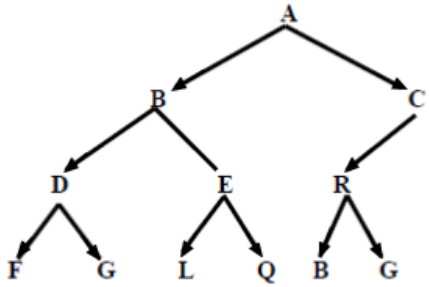
complete binary tree



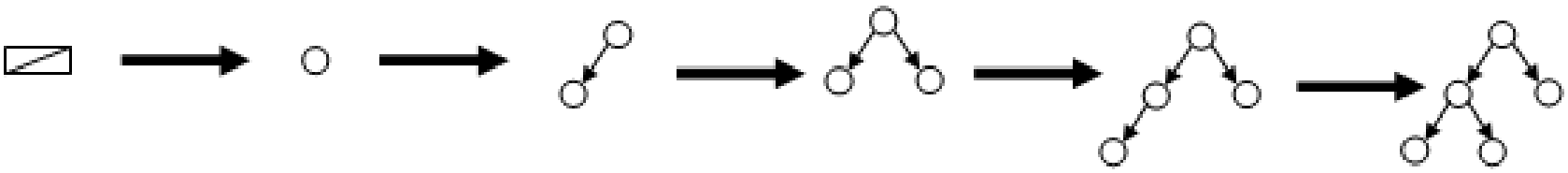
complete binary tree



Not complete binary tree



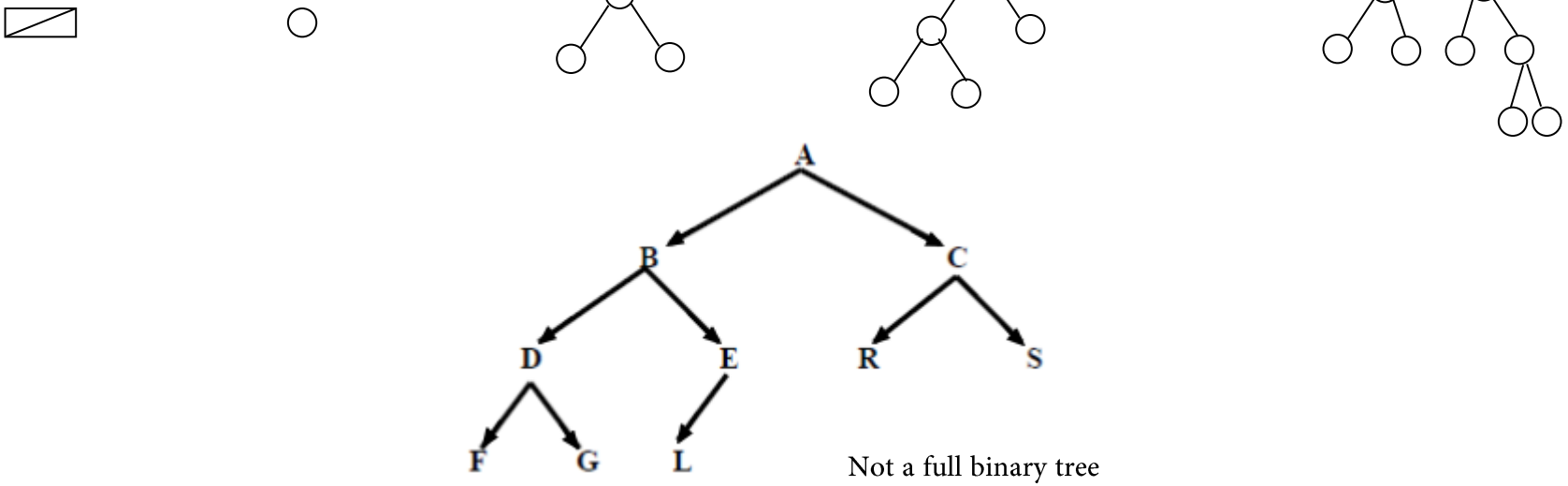
- Example showing the growth of a complete binary tree:



Types of Binary Trees (Contd.)

- A full binary tree is a binary tree in which every node is either a leaf node or an internal node with two children.
  - Every node has either: 0 children, or 2 children

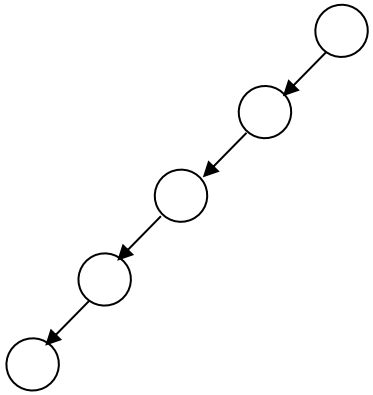
- Theorem: The number of leaves in a non-empty decision tree is one more than the number of internal nodes.



Not a full binary tree

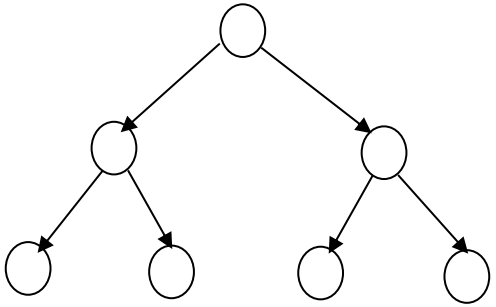
Properties of Binary Trees

- For a binary tree of height h:
  - What is the minimum number of nodes? For which tree?
    - When the tree is linear (Skewed Tree), each node has one child

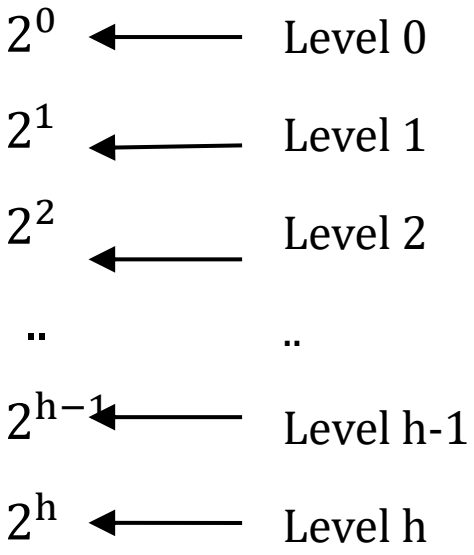


$$h + 1$$

  - What is the maximum number of nodes? For which tree?
    - When the tree is perfect. Each level k, has 2^k node.
    - the maximum number of node is



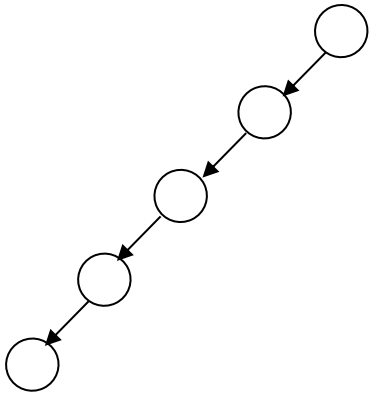
$$\sum_{i=0}^{h} 2^i = 2^{h+1} - 1$$



Properties of Binary Trees

- For a binary tree with n nodes
  - What is the maximum height of a binary tree with n nodes?
    - When the tree is linear, each node has one child

$$n - 1$$

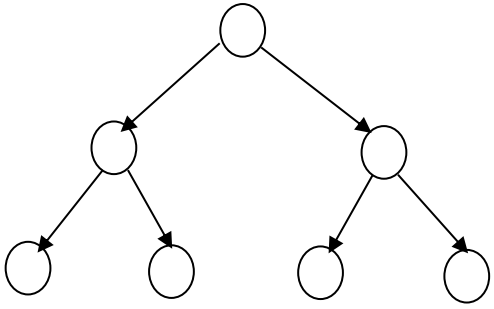


    - What does the tree look like?

  - What is the minimum height of a binary tree with n elements?
    - The minimum height is when the tree is almost complete/perfect. Having the maximum number of node $n = 2^{h+1} - 1$

$$\text{Minimum height } h = \lceil \lg(n+1) \rceil - 1$$

    - What does the tree look like?



Level (Depth) of a node v: The number of edges from the root to node v.

Height of a node v: The number of edges from v to the farthest leaf

Binary Tree Implementation

- Linked Representation
  - Each node contains:

left	key	right

```
public class BTNode<T extends Comparable<T>> {
   protected T key;
   protected BTNode<T> left, right;

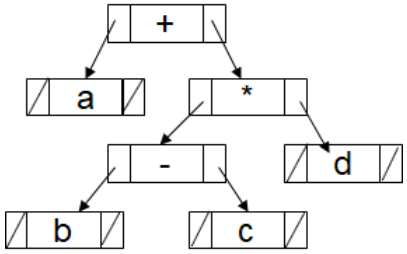
    public BTNode() {
       this.left = this.right = null;
    }

   public BTNode(T key) {
      this(key,null,null);
   }

   public BTNode(T key, BTNode<T> left, BTNode<T> right) {
      this. key = key;
      this.left = left;
      this.right = right;
   }
}
```

Example: A binary tree representing

a + (b - c) * d




```
public class BinaryTree<T extends Comparable<T>> {
   BTNode<T> root;

public BinaryTree() {
      this.root = null;
}

public BinaryTree(BTNode<T> root) {
      this.root = root;
}

public void clear(){
      this.root = null;
}

public boolean isEmpty(){
     return this.root == null;
}

public void insert(T key){…}

public void delete (T data){…}

public void inorderTraversal(){ …. }

public void preorderTraversal(){ … }

public void postorderTraversal(){ … }

public void breadthorderTraversal(){ … }

public boolean search(T key){…}
```

Binary Tree: Traversals, Insertions, and Deletions

- Tree Traversal classification
  - BreadthFirst traversal
  - DepthFirst traversals: Preorder, Inorder, and Postorder
- Traversing Expression Trees
- Insertion in a Binary tree
- Deletion from a Binary tree

Tree Traversal

- The process of systematically visiting all the nodes in a tree and performing some processing at each node in the tree is called a tree traversal.
  - A traversal starts at the root of the tree and visits every node in the tree exactly once.

- There are two common methods in which to traverse a tree:
  - Breadth-First Traversal (or Level-order Traversal).
  - Depth-First Traversal:
    - Preorder traversal
    - Inorder traversal (for binary trees only)
    - Postorder traversal

Breadth-First Traversal

- Initialize the queue by enqueuing the root node of the tree to traverse
- While the queue is not empty:
  - Dequeue a node from the queue,
  - Output the data field of this node,
  - Enqueue the left child of this node to the queue if the child is not empty.
  - Enqueue the right child of this node to the queue if the child is not empty.

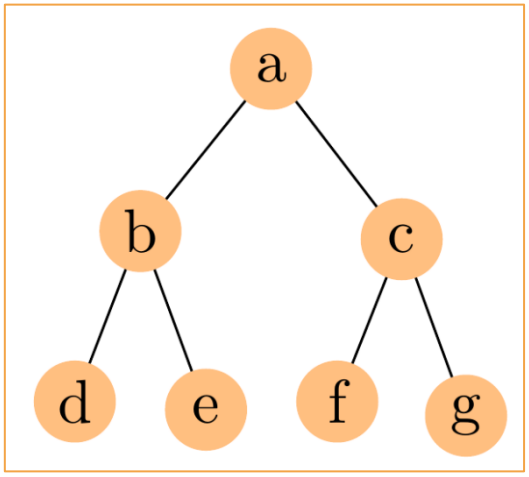
```
public void levelOrderTraversal() {
    if(this.root==null)
        return;
    BSTNode<T> tmp ;
    Queue<BSTNode<T>> queue = new LinkedList<>();

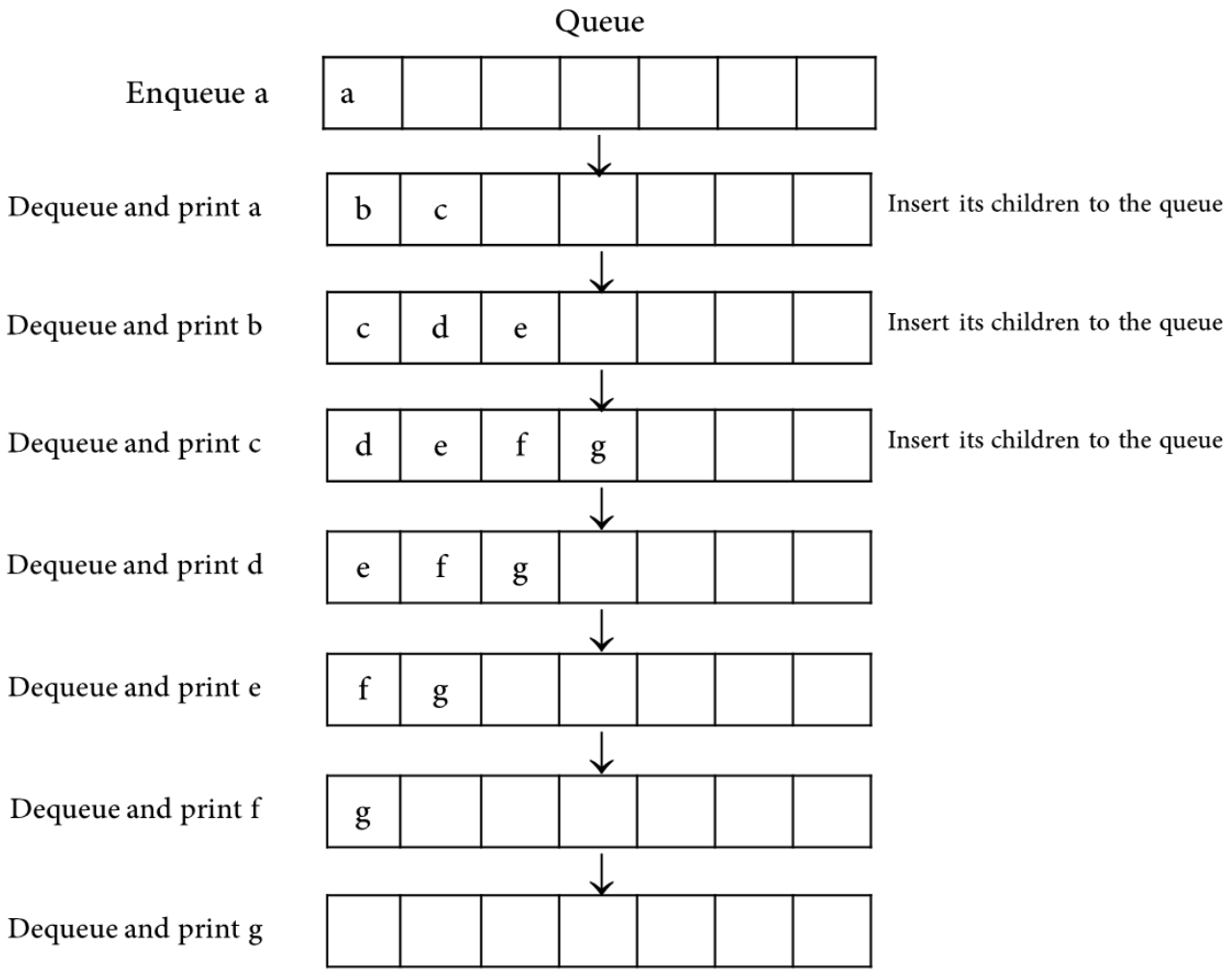
    queue.add(root);
    while (!queue.isEmpty()) {
        tmp = queue.poll();
        System.out.println(tmp.key);

        if (tmp.left != null)
            queue.add(tmp.left);

        if (tmp.right != null)
            queue.add(tmp.right);
    }
}
```

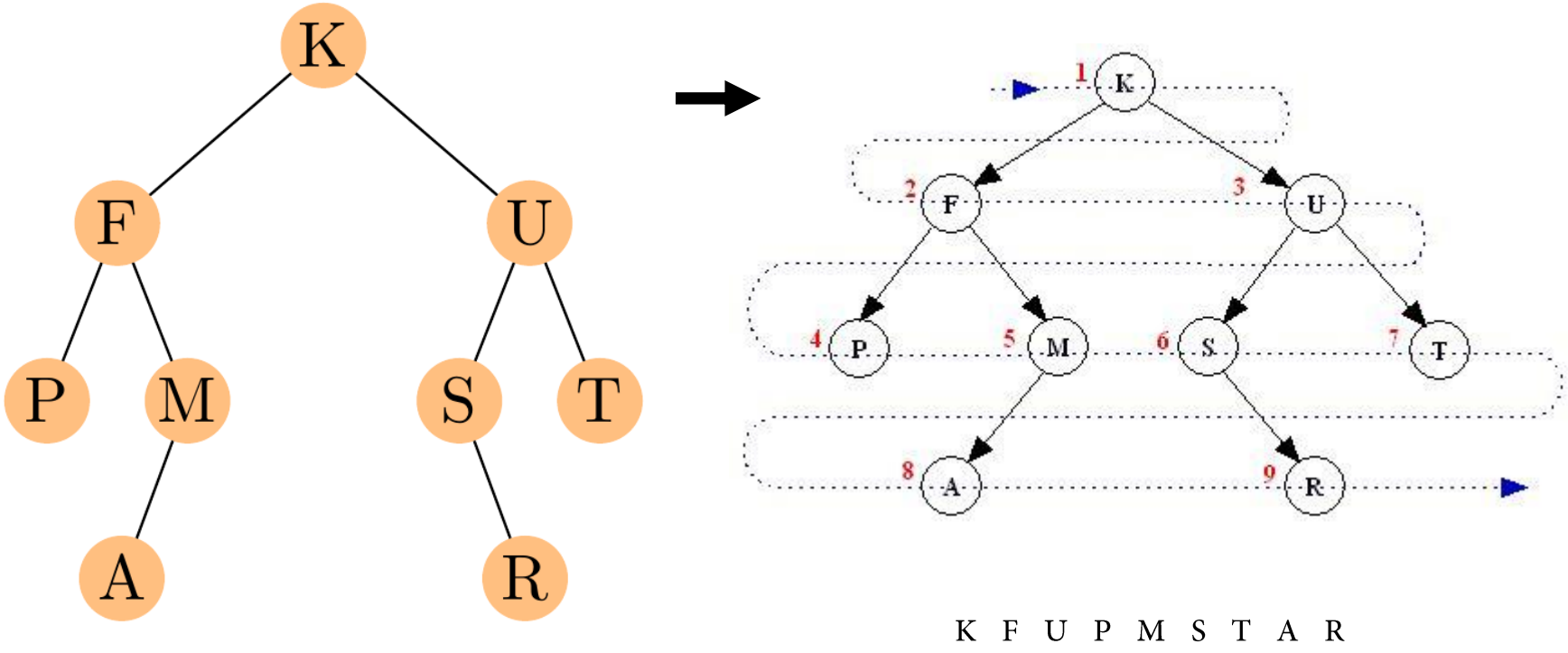
Breadth-First Traversal (Contd.)





Breadth-First Traversal (Contd.)

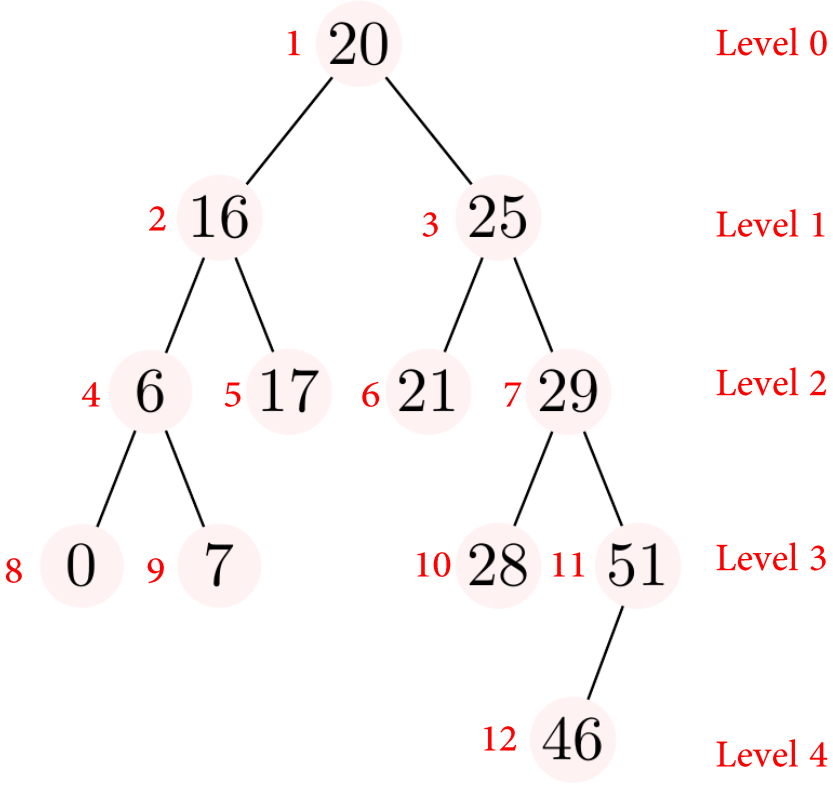
- Breadth-First traversal visits a tree level-wise from top to bottom
  - Visit each level: 0, 1, 2, 3 and 4
  - Within each level, visit each node from left to right



Breadth-First Traversal (Contd.)

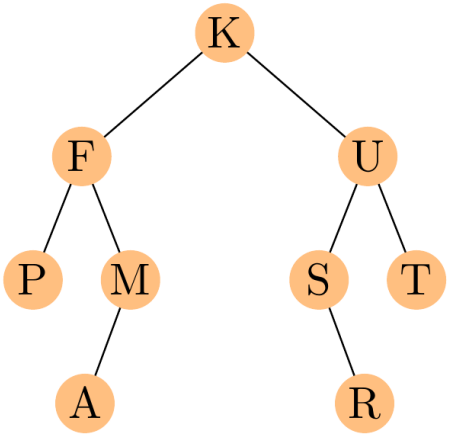
- Visit each level: 0, 1, 2, 3 and 4
- Within each level, visit each node from left to right

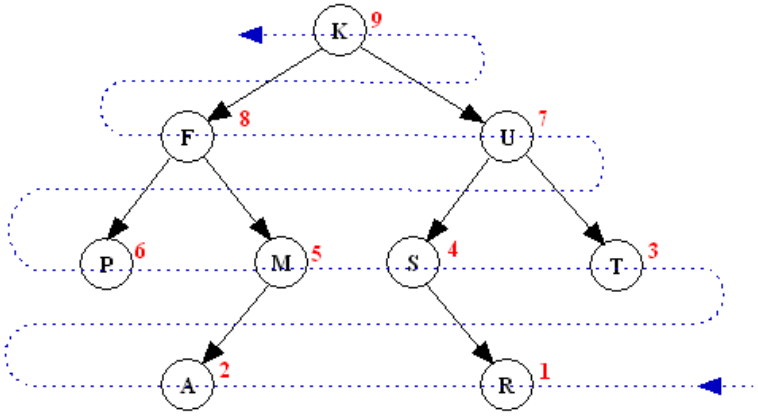
20, 16, 25, 6, 17, 21, 29, 0, 7, 28, 51, 46



Breadth-First Traversal (Contd.)

- Exercise: Write a BinaryTree instance method for Reverse Breadth-First Traversal





R A T S M P U F K

```
public void levelOrderTraversal() {
    if(this.root==null)
        return;
    BSTNode<T> tmp ;
    Queue<BSTNode<T>> queue = new LinkedList<>();

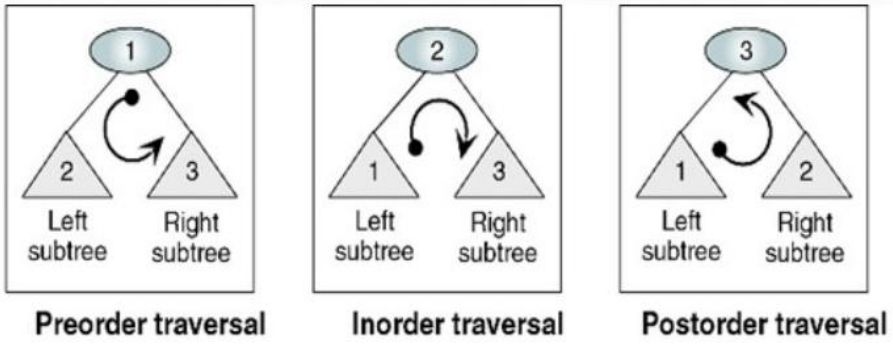
    queue.add(root);
    while (!queue.isEmpty()) {
        tmp = queue.poll();
        System.out.println(tmp.key);

        if (tmp.left != null)
            queue.add(tmp.left);

        if (tmp.right != null)
            queue.add(tmp.right);
    }
}
```

Modify

Depth-first Traversal



Pre ⟶ Node comes before its children

In ⟶ Node comes in between its children

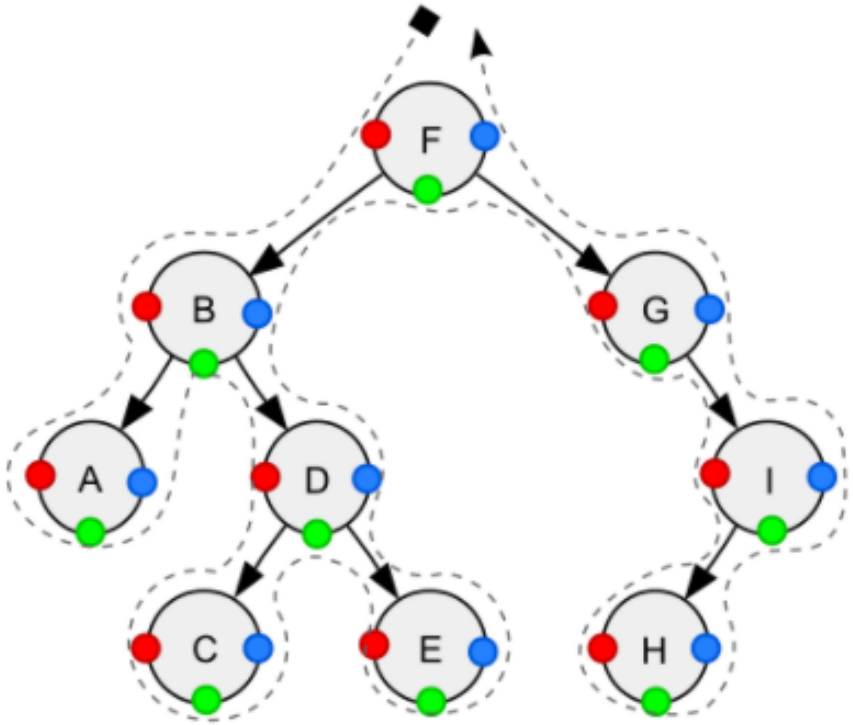
Post ⟶ Node comes after its children

When performing recursive traversal of a binary tree, each node is conceptually passed three times:

First time – when we arrive at the node.

Second time – after finishing the left subtree.

Third time – after finishing the right subtree.



Depth-first traversal (dotted path) of a binary tree:

Pre-order (node access at position red ●):

F, B, A, D, C, E, G, I, H;

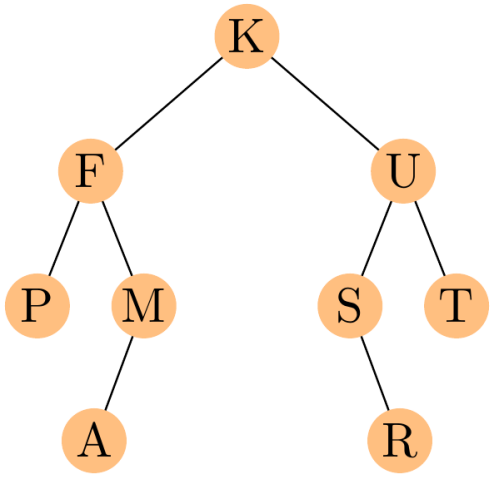
In-order (node access at position green ●):

A, B, C, D, E, F, G, H, I;

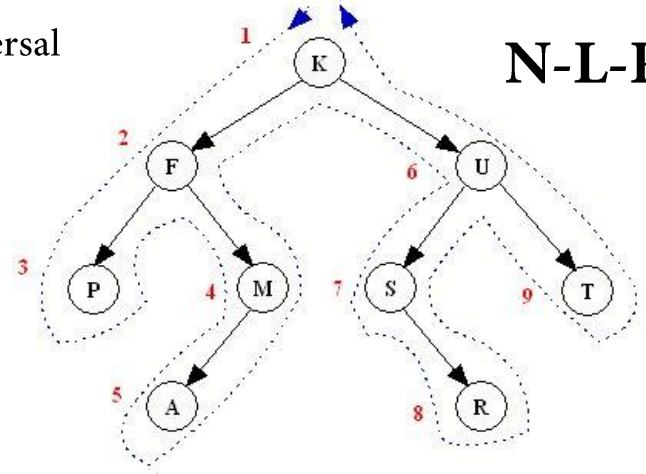
Post-order (node access at position blue ●):

A, C, E, D, B, H, I, G, F.

Depth-first Traversal

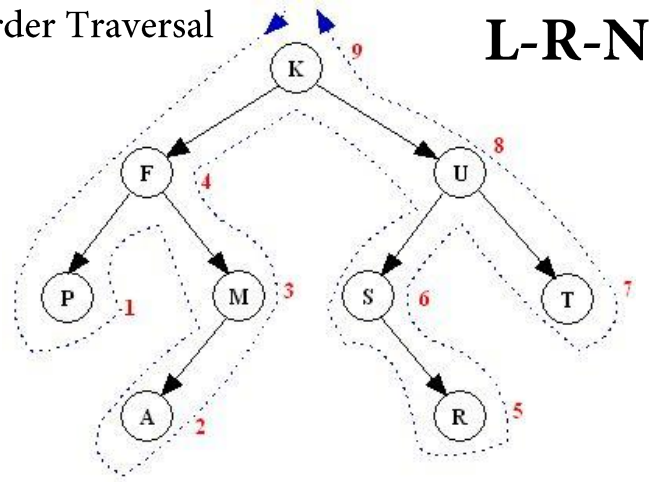


Preorder Traversal **N-L-R**



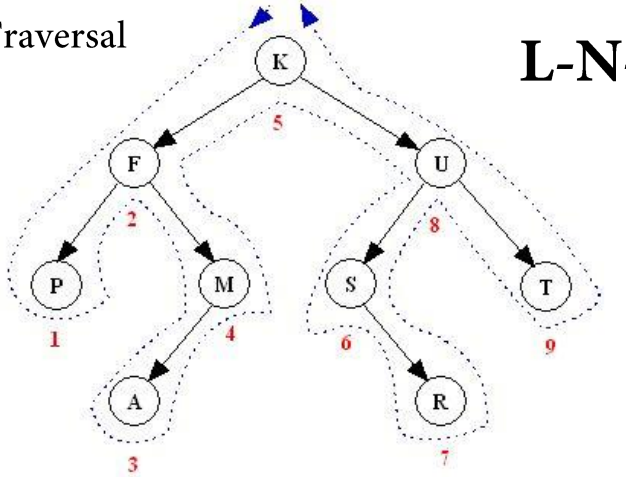
K F P M A U S R T

Postorder Traversal **L-R-N**



P A M F R S T U K

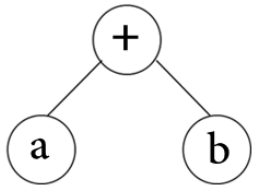
Inorder Traversal **L-N-R**



P F A M K S R U T

Depth-First Traversals

Name	for each Node:	CODE

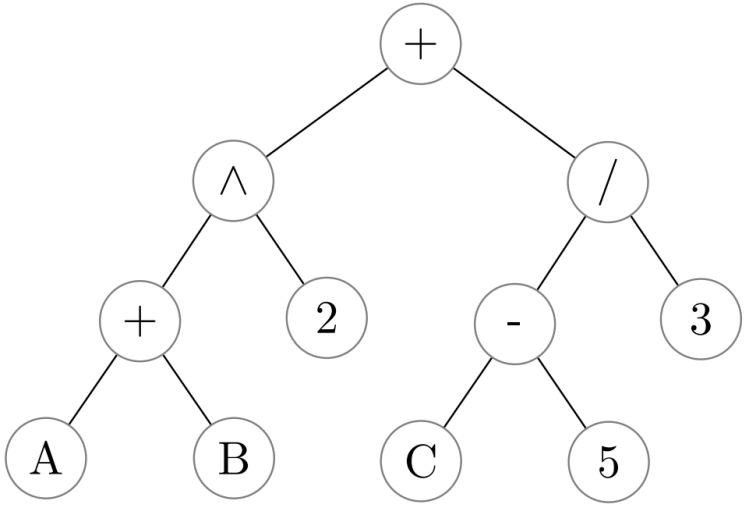
Preorder (N-L-R)	Visit the node
	Visit the left subtree, if any.
	Visit the right subtree, if any.
		void preorder (Node node) {
		if (node == null) return;
		System.out.print(node.key + " ");
		preorder (node.left);
		preorder (node.right);
		}
		 
		
		+ a b

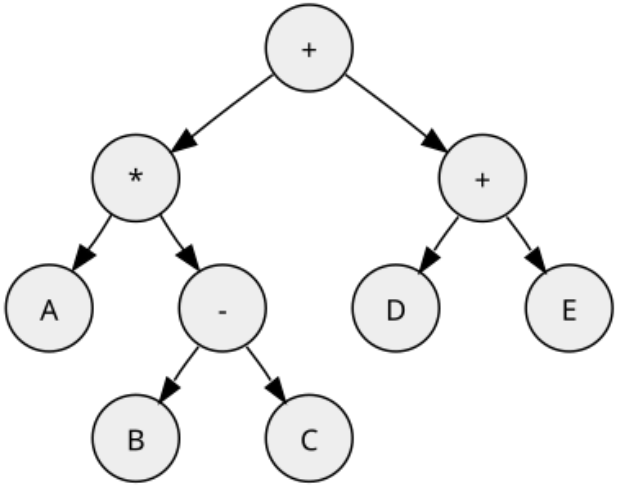
Inorder (L-N-R)	Visit the left subtree, if any.
	Visit the node
	Visit the right subtree, if any.
		void inorder (Node node) {
		if (node == null) return;
		inorder (node.left);
		System.out.print(node.key + " ");
		inorder (node.right);
		}
		a + b

Postorder (L-R-N)	Visit the left subtree, if any.
	Visit the right subtree, if any.
	Visit the node
		void postorder (Node node) {
		if (node == null) return;
		postorder (node.left);
		postorder (node.right);
		System.out.print(node.key + " ");
		}
		a b +

Traversing Expression Trees

- An inorder traversal produces the original expression(without parentheses), in infix order
- A preorder traversal produces a prefix expression
- A postorder traversal produces a postfix expression





Tree representing the arithmetic expression: $A * (B - C) + (D + E)$

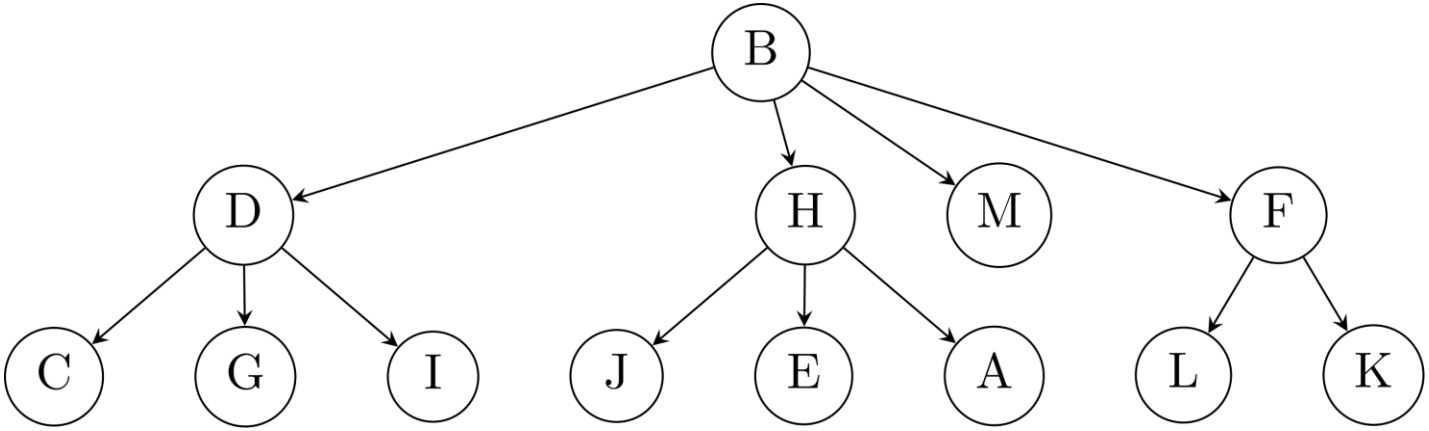
Infix: A + B ^ 2 + C – 5 / 3

Prefix: + ^ + A B 2 / - C 5 3

Postfix: A B + 2 ^ C 5 - 3 / +

An expression tree for ((A + B) ^ 2) + ((C – 5) / 3)

Traversing N-ary/General Trees



Preorder: BDCGIHJEAMFLK

Inorder: CDGIBJHEAMLFK

Postorder: CGIDJEAHMLKFB

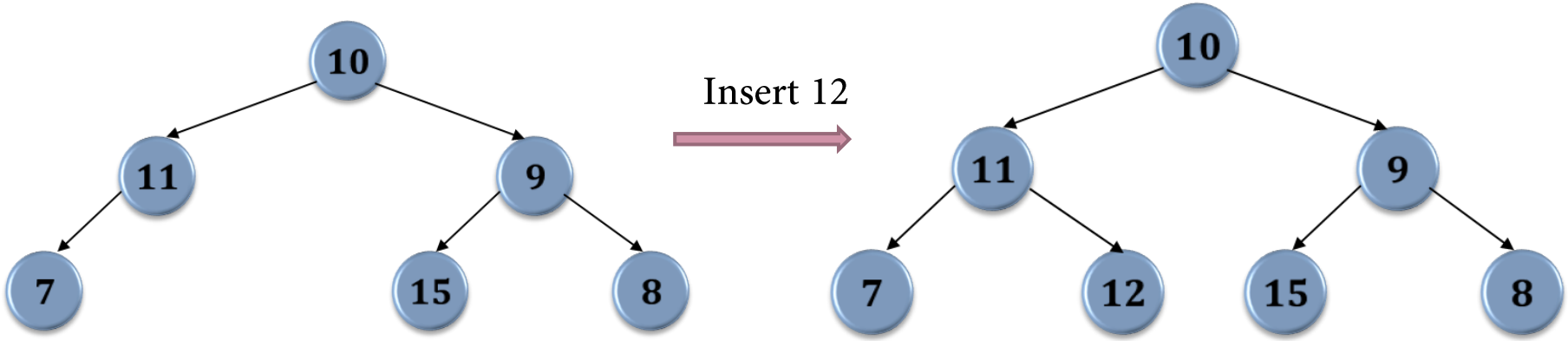
Binary-tree Insertion and deletion

- There are no fixed rules for inserting and deleting from a Binary-tree. In our Binary-tree implementation, we use the insertion and deletion algorithms given below:

```
if (node.left == null) {
    node.left = new Node(key);
    break; }
  else {
     queue.add(node.left);
  }
```

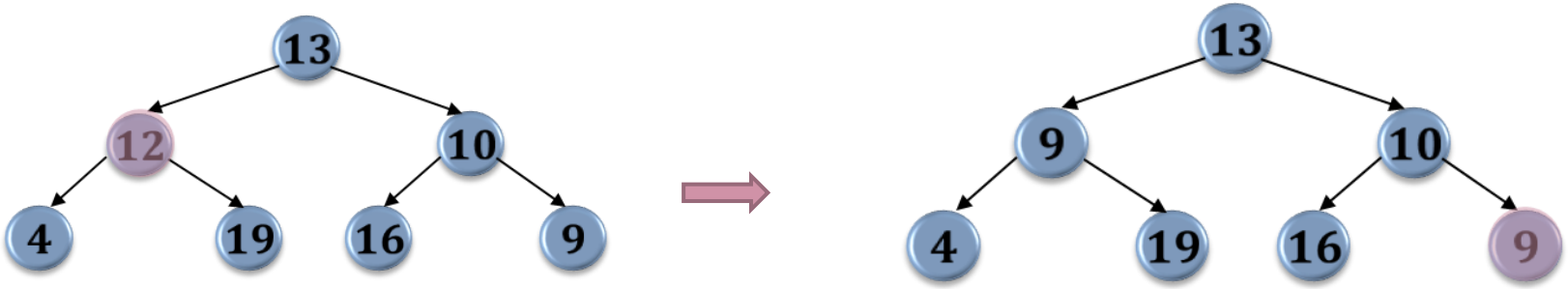
```
q.add(root);
while(!q.isEmpty()){
    BTNode<T> n =  q.remove();
    System.out.print(" " + n.key);
    if(n.left!=null)
      q.add(n.left);
    if(n.right!=null)
      q.add(n.right);
}
```

  - Given a binary tree and a key,
  - insert the key into the binary tree
  - at the first position available in level order traversal.



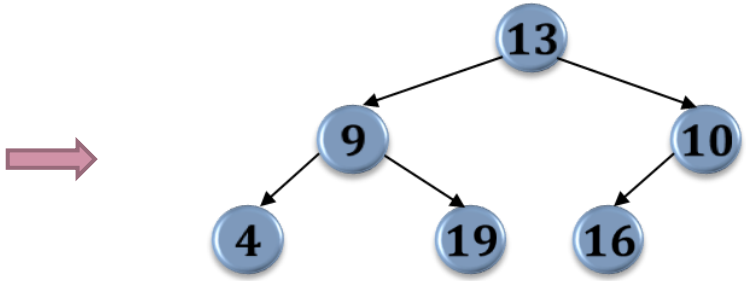
Binary-tree Insertion and deletion

- Given a binary tree, delete a node from it by making sure that tree shrinks from the bottom (i.e. the deleted node is replaced by the last leaf node).



Node to be deleted is 12

Replace 12 by last leaf node value

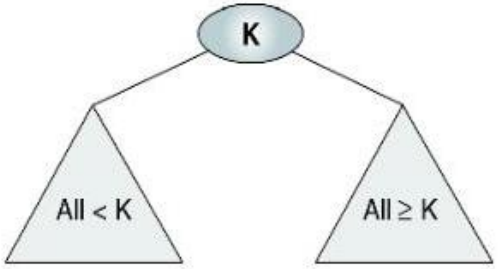


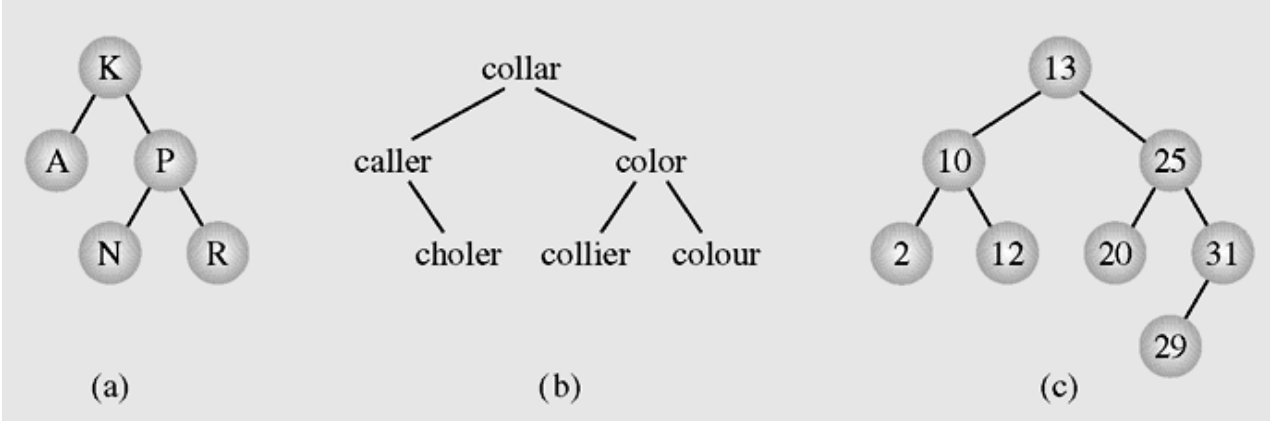
Binary Search Trees (BSTs)

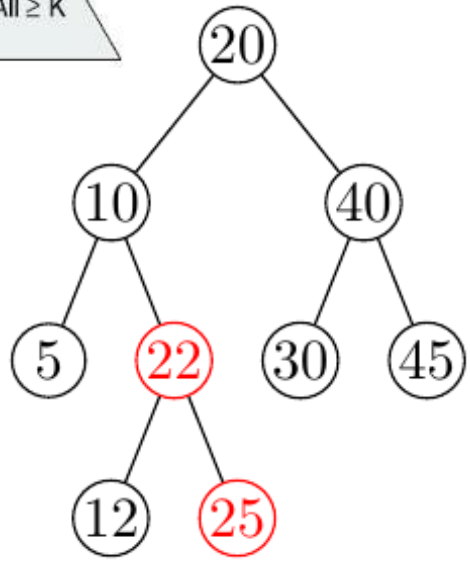
- What is a Binary search tree?
- Binary search tree implementation
- Binary search tree inorder traversals
- Insertion in a BST
- Deletion from a BST
- BSTs as Priority Queues
- Why Binary search trees?
- Disadvantages of Binary search trees

Binary Search Tree (Definition)

- A binary search tree (BST) is a binary tree that is empty or that satisfies the BST ordering property:
  1. The key of each node is greater than each key in the left **subtree**, if any, of the node.
  2. The key of each node is less than each key in the right **subtree**, if any, of the node.
- Thus, each key in a BST is unique.
- Examples:







not a BST because

Binary Search Tree Implementation

- The BinarySearchTree class is a subclass of the BinaryTree class:

```
public class BST<T extends Comparable<T>> extends BinaryTree
```

- It uses the BTNode class

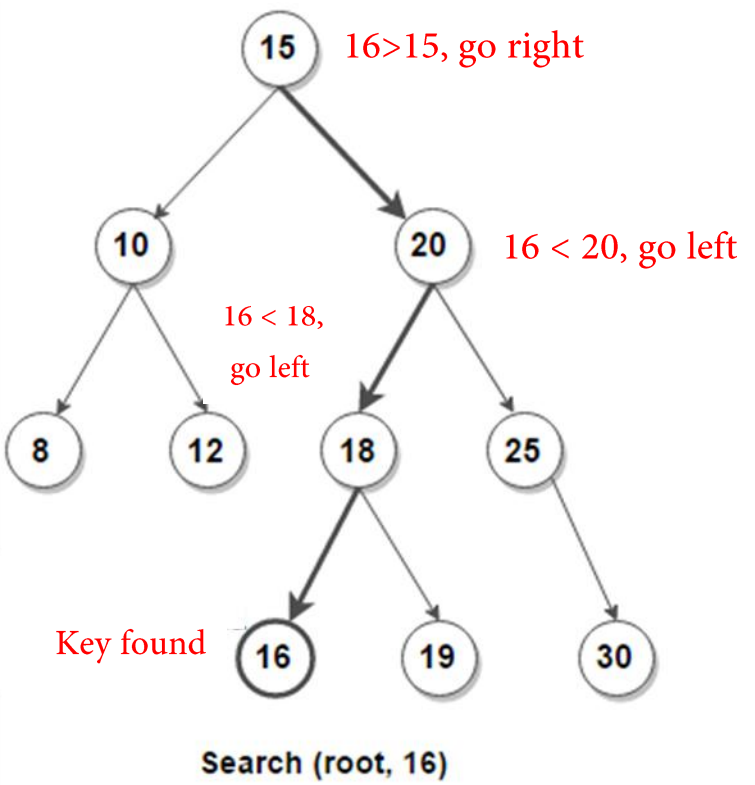
- It inherits the methods isEmpty, purge, getHeight, printBinaryTree, and the traversal methods of BinaryTree.

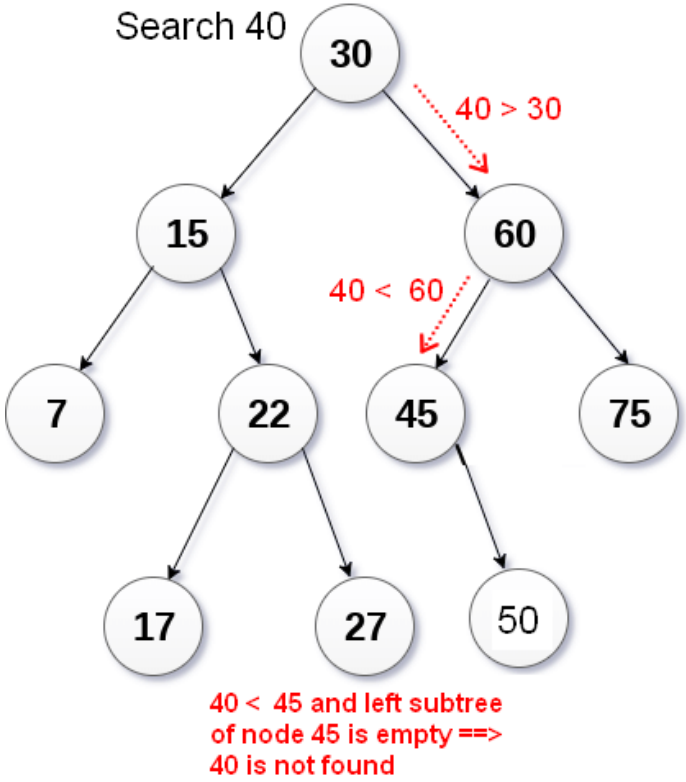
- It overrides the BinaryTree methods insert, search and deleteByCopying.

- It defines its own methods: deleteByMerging, recursiveInsert, searchRecursive, getMin, getMax, getInorderSuccessor, getInorderPredecessor, treeSortAscending, and treeSortDescending

Binary Search Tree Implementation: search

Search 16





Binary Search Tree Implementation: search

1. Compare the target with the root key of the tree.
   1. If the item is matched then return the location of the node.
   2. Otherwise check if target is LESS THAN the root key, if so then move to the left sub-tree.
   3. If not, then move to the right sub-tree.
2. Repeat this procedure until a match found or it is not found.
3. If element is not found then return false, or null or throw an exception.

```
public T find(T key) {
      BSTNode<T> p = this.root;
      while (p != null)
         if (key.equals(p.key))
            return p.key;
         else if (key.compareTo(p.key) < 0)
            p = p.left;
         else p = p.right;
      return null;
   }
```

The iterative search method of the BinarySearchTree class:

```
public boolean searchRecursive(T key) {
      return searchRecursive(root, key);
   }
private boolean searchRecursive(BTNode current, T key) {
      if (current == null)
         return false;
      int comparisonResult = key.compareTo(current.ksy);
      if (comparisonResult == 0)
         return true;
       else if (comparisonResult < 0)
         return searchRecursive(current.left, key);
       else
         return searchRecursive(current.right, key);
   }
```

Insertion and Search

- By the BST ordering property, a new node is always inserted as a leaf node.
- The recursive insert method recursively finds an appropriate empty subtree to insert the new key:

```
public boolean searchRecursive(T key) {
    return searchRecursive(root, key);
  }

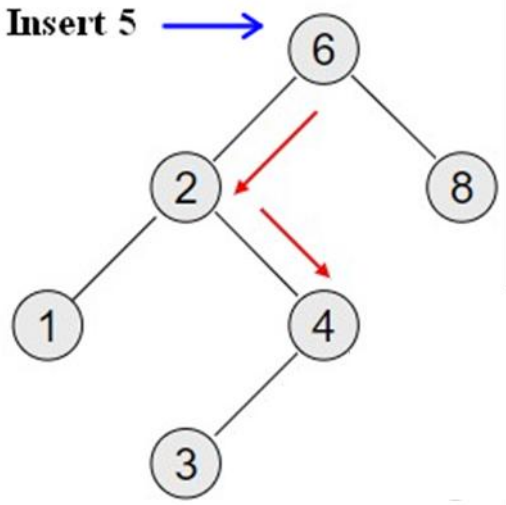
private boolean searchRecursive(BTNode current, T key) {
    if (current == null)
      return false;

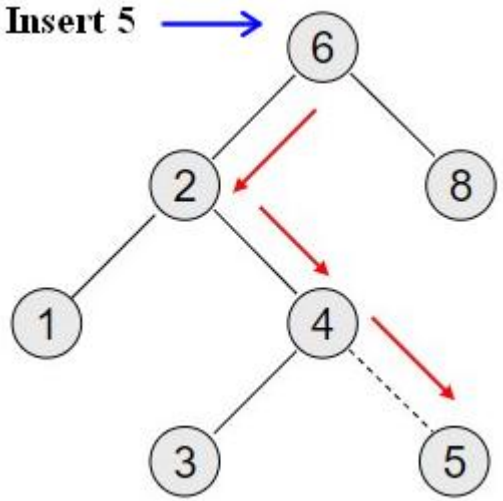
    int comparisonResult = key.compareTo(current.key);
    if (comparisonResult == 0)
      return true;
     else if (comparisonResult < 0)
      return searchRecursive(current.left, key);
     else
      return searchRecursive(current.right, key);
  }
```

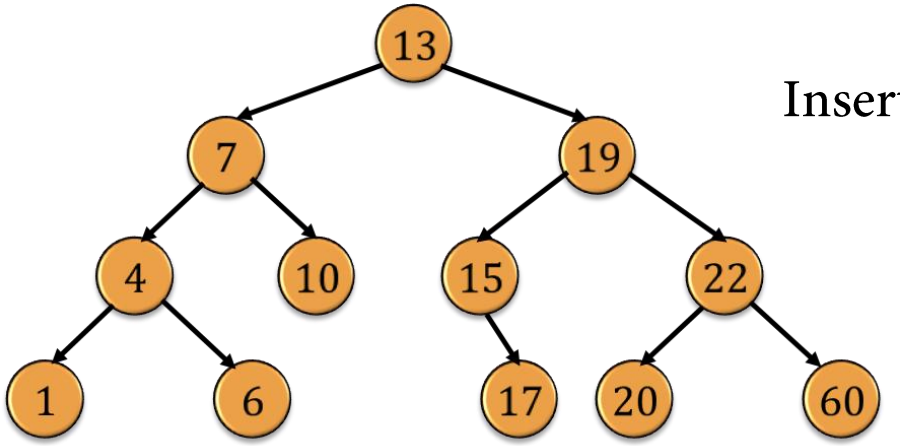
```
public void recursiveInsert(T key) {
   root = recursiveInsert(root, key);
 }
private BTNode<T> recursiveInsert(BTNode<T> p, T key) {
   if(p == null)
     p = new BTNode<T>(key);
    else{
     int comparisonResult = key.compareTo(p.key);
     if(comparisonResult == 0)
       throw new IllegalArgumentException("Duplicate key.");
     else if(comparisonResult < 0)
         p.left  = recursiveInsert(p.left, key);
      else
        p.right = recursiveInsert(p.right, key);
    }
     return p;
 }
```

Insertion in a BST (Contd.)

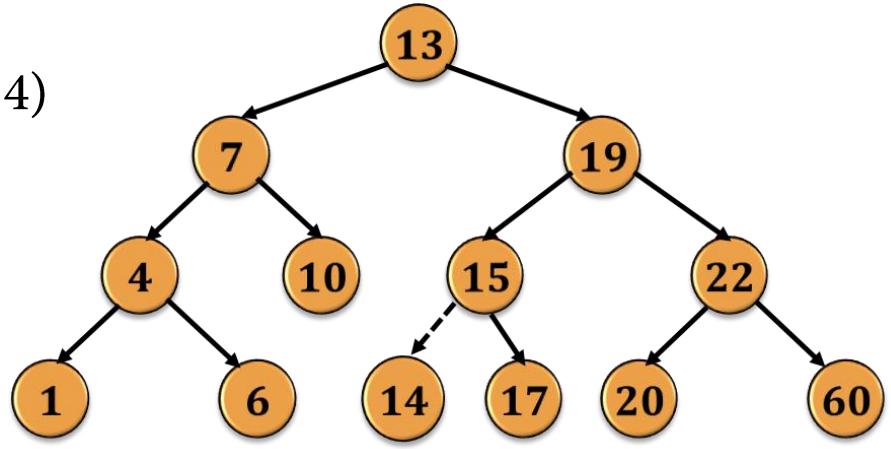
- Search and insertion order determine insertion point



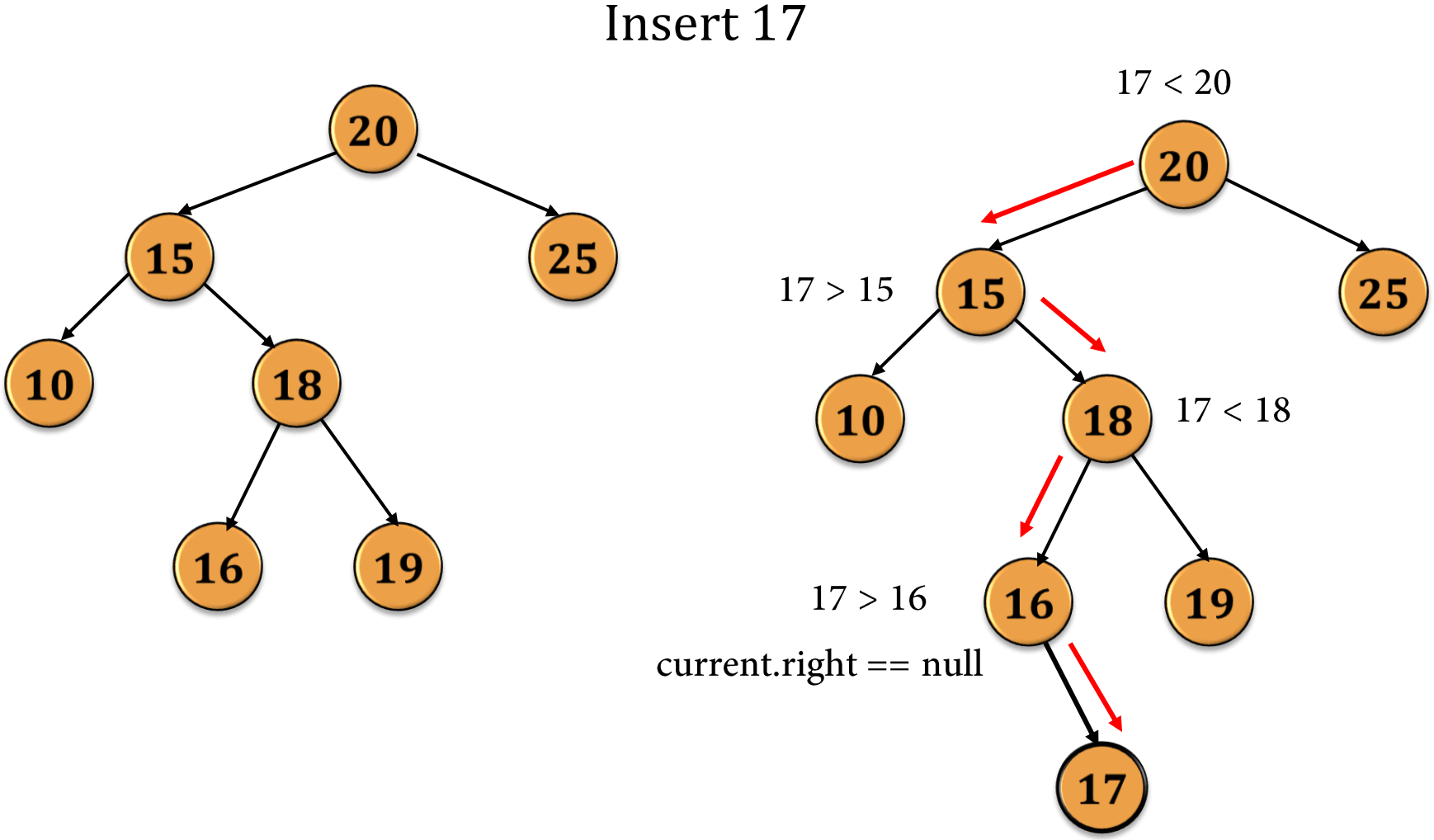




Insert (14)

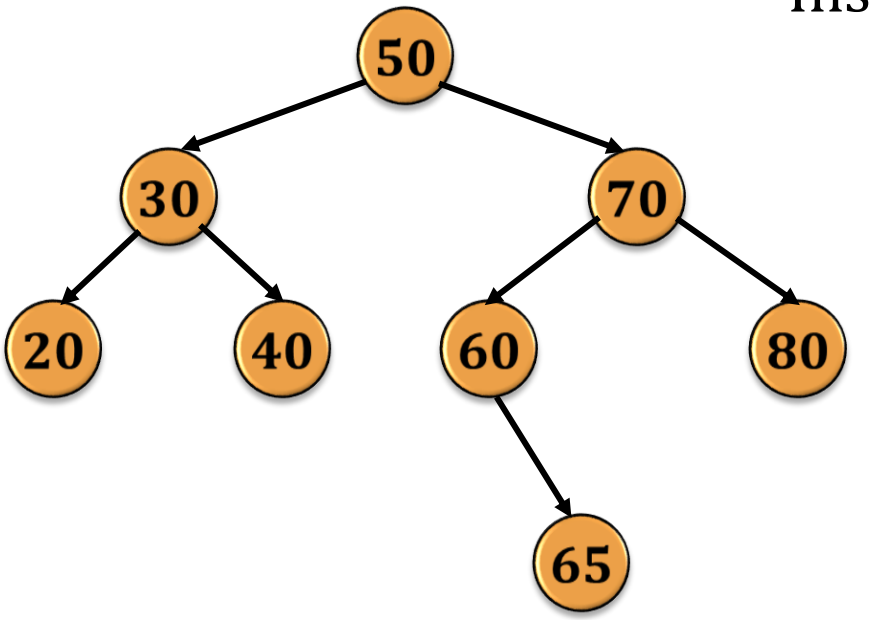


Insertion in a BST (Contd.)

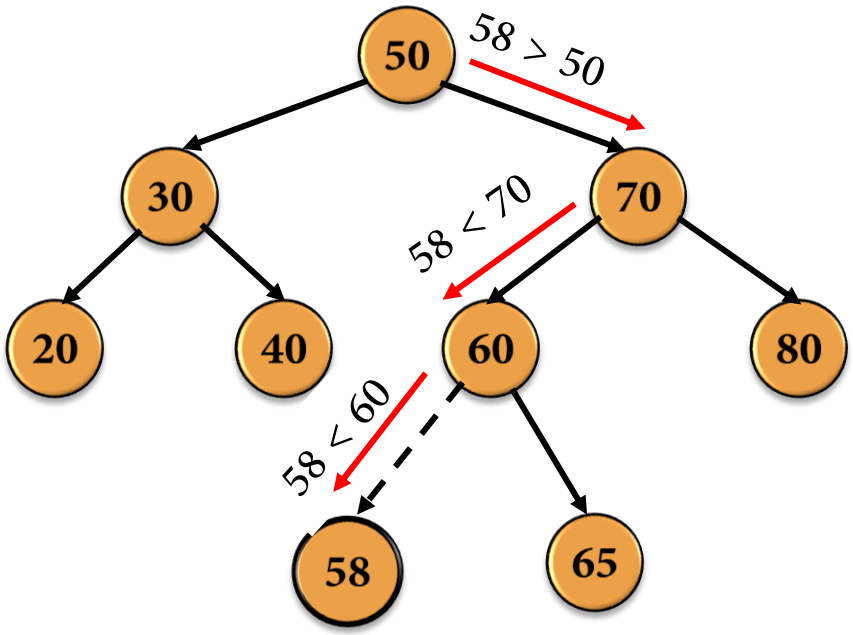


Insertion in a BST (Contd.)

- Search and insertion order determine insertion point



Insert 58



current.left == null

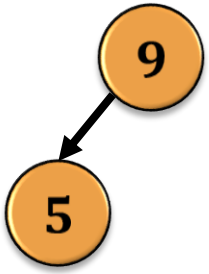
Insertion in a BST (Contd.)

- **Construct a Binary Search Tree (BST) by inserting the following values in order: 9, 5, 10, 6, 3, 4, 2**

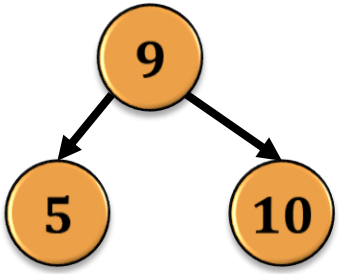
Insert 9



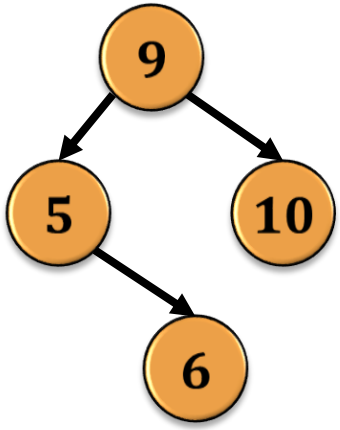
Insert 5



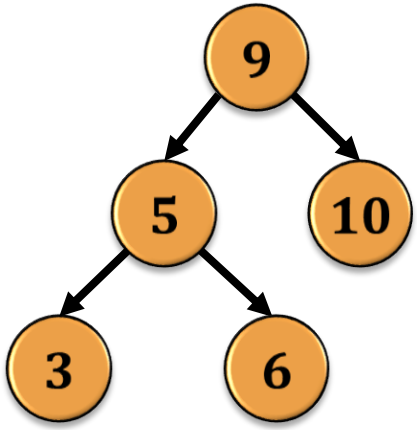
Insert 10



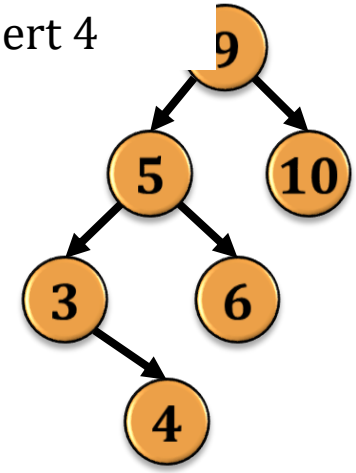
Insert 6



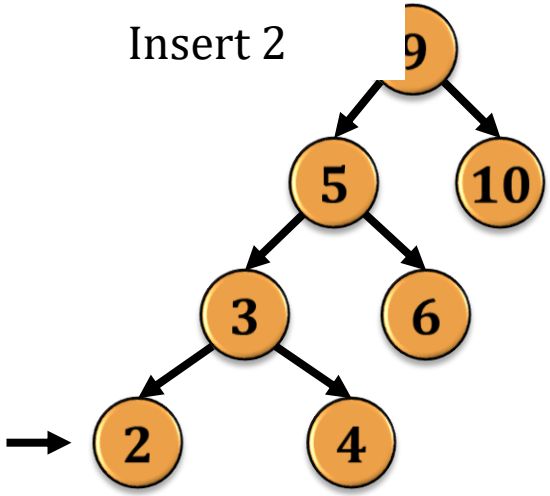
Insert 3



Insert 4

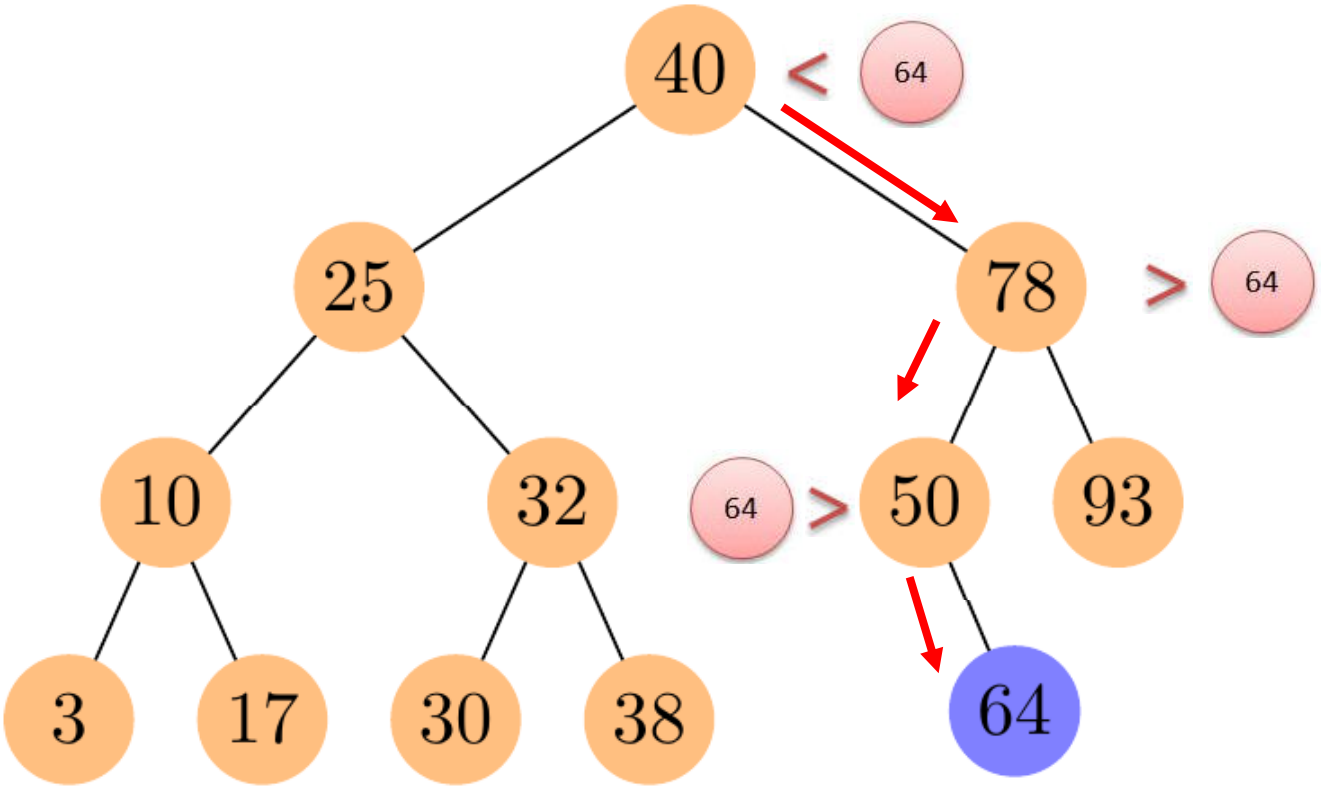


Insert 2



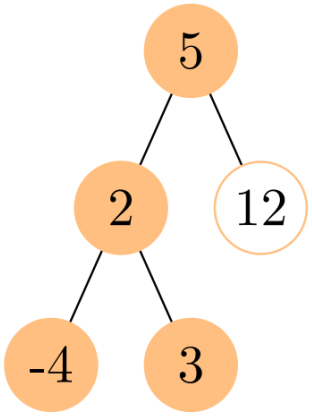
Insertion in a BST (Contd.)

- Search and insertion order determine insertion point

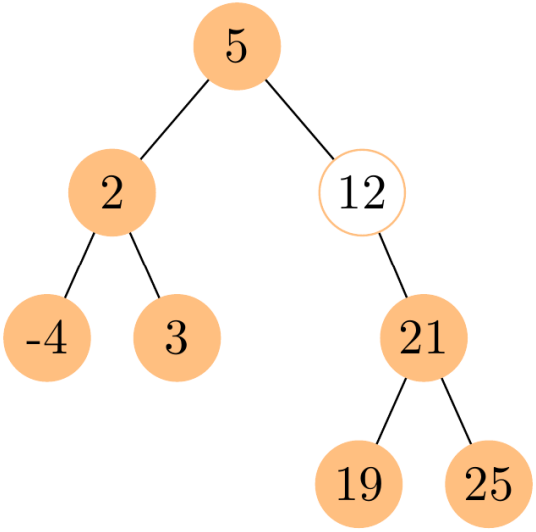


Deletion in a BST

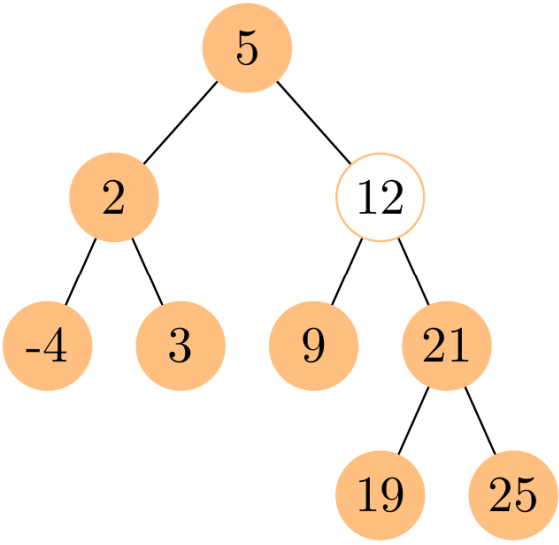
- There are three cases:
  - The node to be deleted is a leaf node.
    - Make null the parent reference to the leaf node to be deleted
  - The node to be deleted has one non-empty subtree.
    - Replace the target with its child
  - The node to be deleted has two non-empty subtrees.



case1



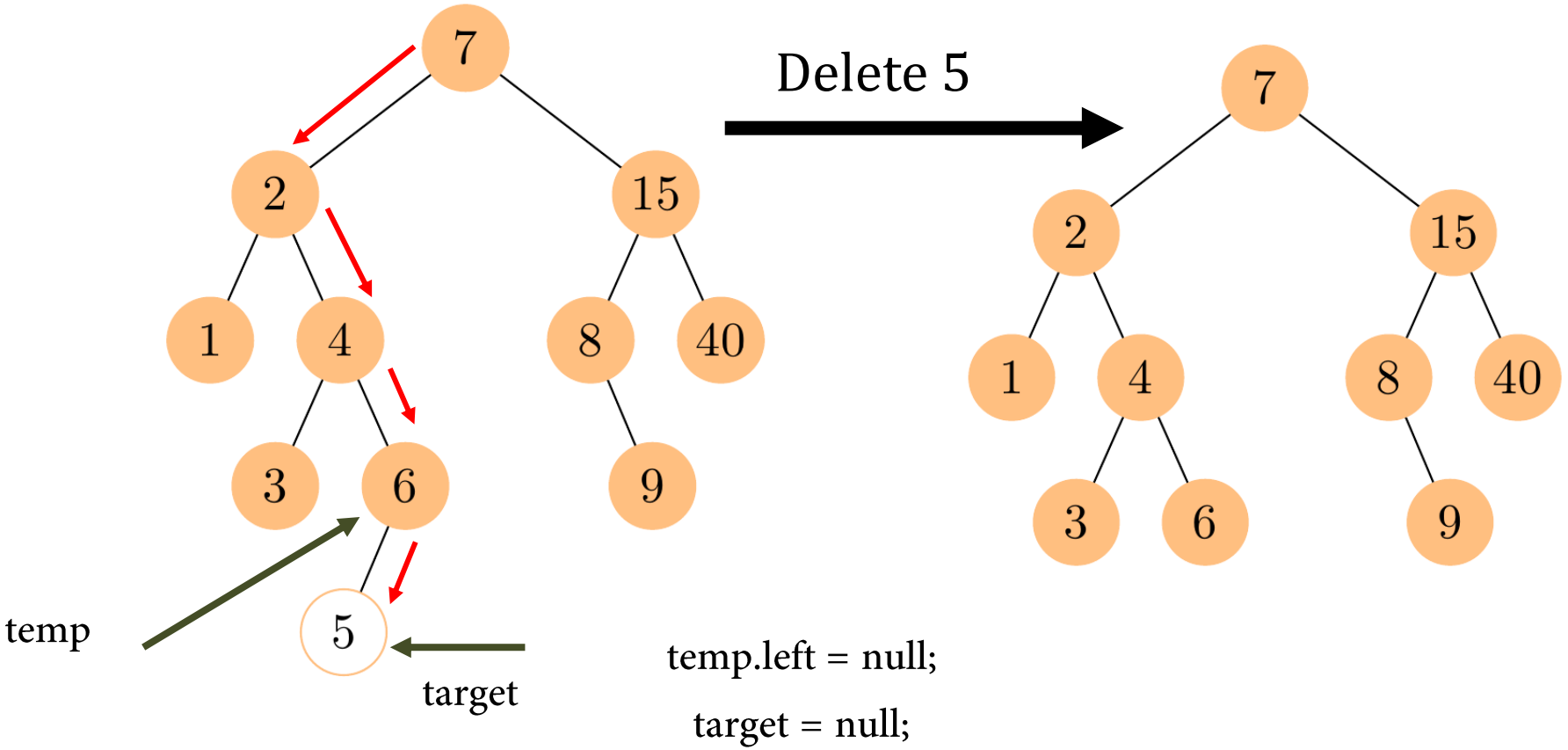
case2



case3

CASE 1: Deleting a Leaf Node

- Make null the parent reference to the leaf node to be deleted.



CASE 2: Deleting a node with one non-empty subtree

- (a) The right subtree of the node x to be deleted is empty.
- (b) The left subtree of the node x to be deleted is empty.

(a)

```
// Let target be a reference to the node x.
BTNode temp = target.left;
target.key = temp.key;
target.left = temp.left;
target.right = temp.right;
temp = null;
```

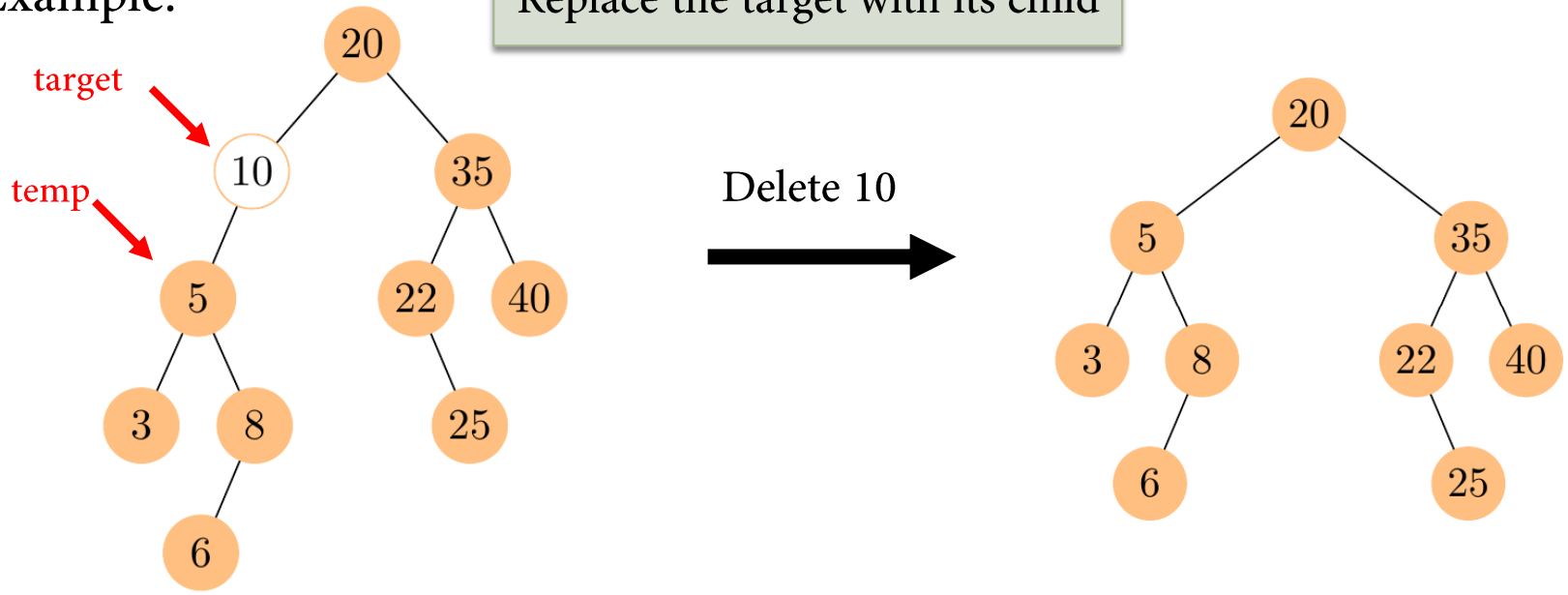
O(1)

```
// Let target be a reference to the node x.
BTNode temp = target.right;
target.key = temp.key;
target.right = temp.rigth;
target.left = temp.left;
temp = null;
```

(b)

- Example:

Replace the target with its child



CASE 3: Deleting a node that has two non-empty subtrees

- Two methods to delete node x:

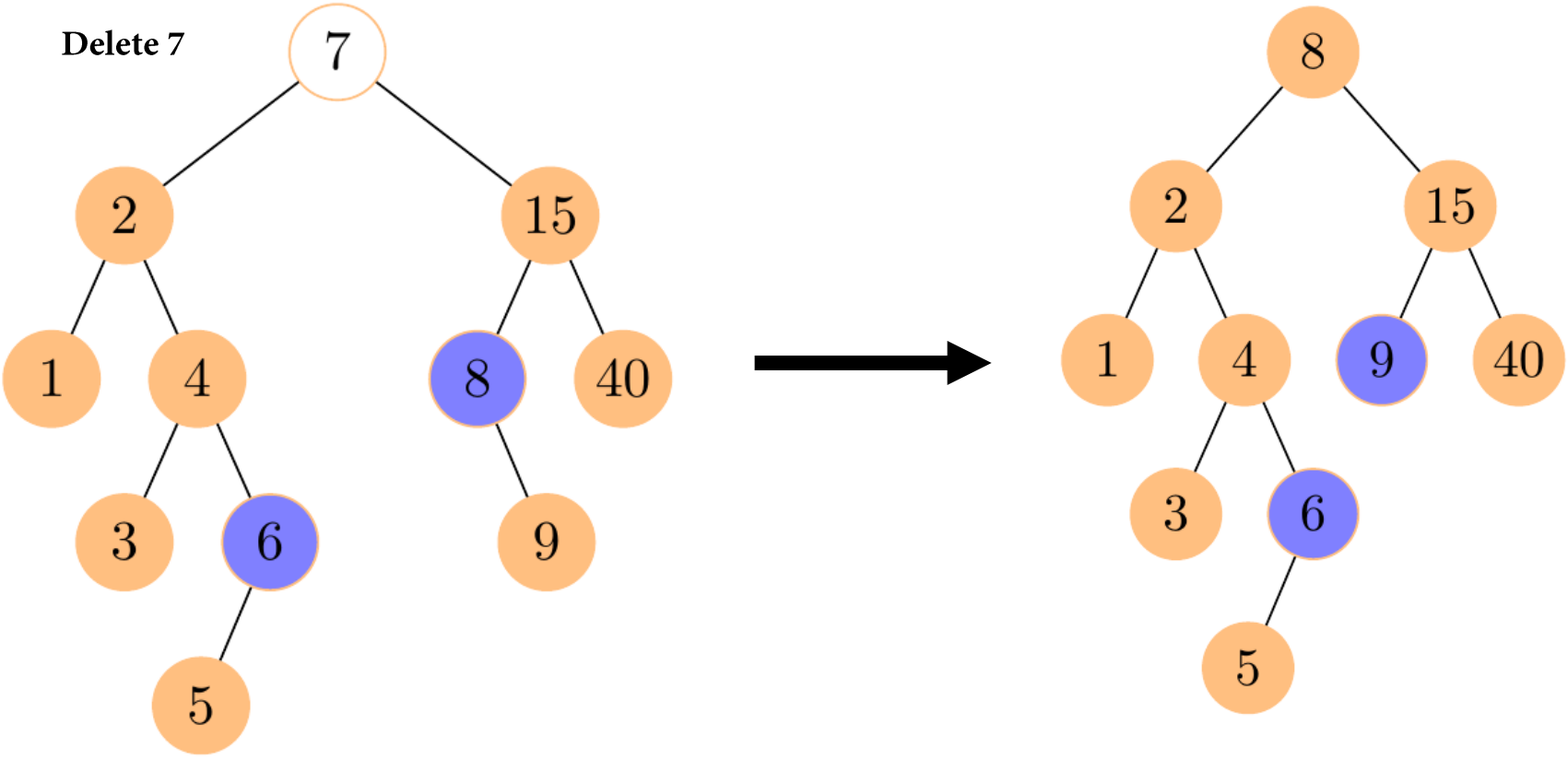
  - Deletion by Copying

    - Replace the key being deleted with either its inorder predecessor (maximum in left subtree) or inorder successor (minimum in right subtree)
      - the minimum key in the right subtree of x, to the node x.
      - the maximum key in the left subtree of x, to the node x
    - Delete that successor node.
      - How many children does that successor node have?

  - Deletion by Merging

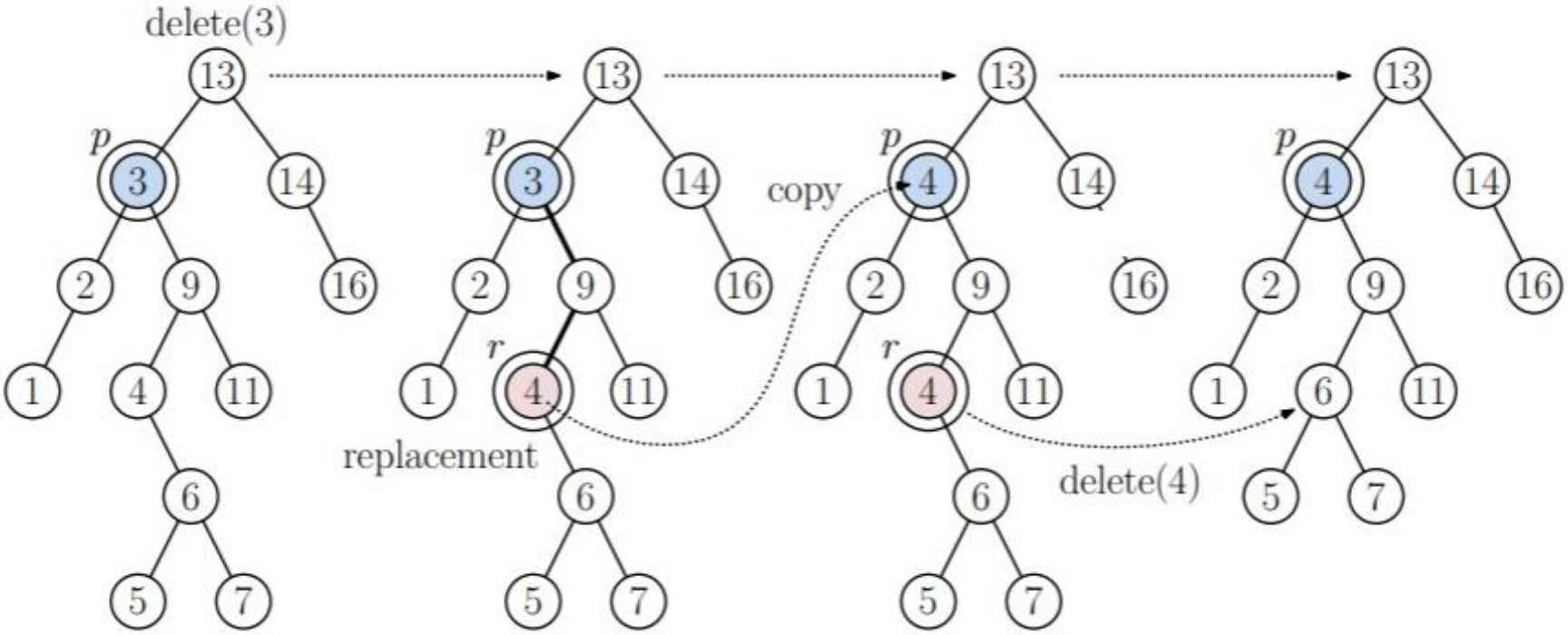
Deletion by Copying

- Example 1



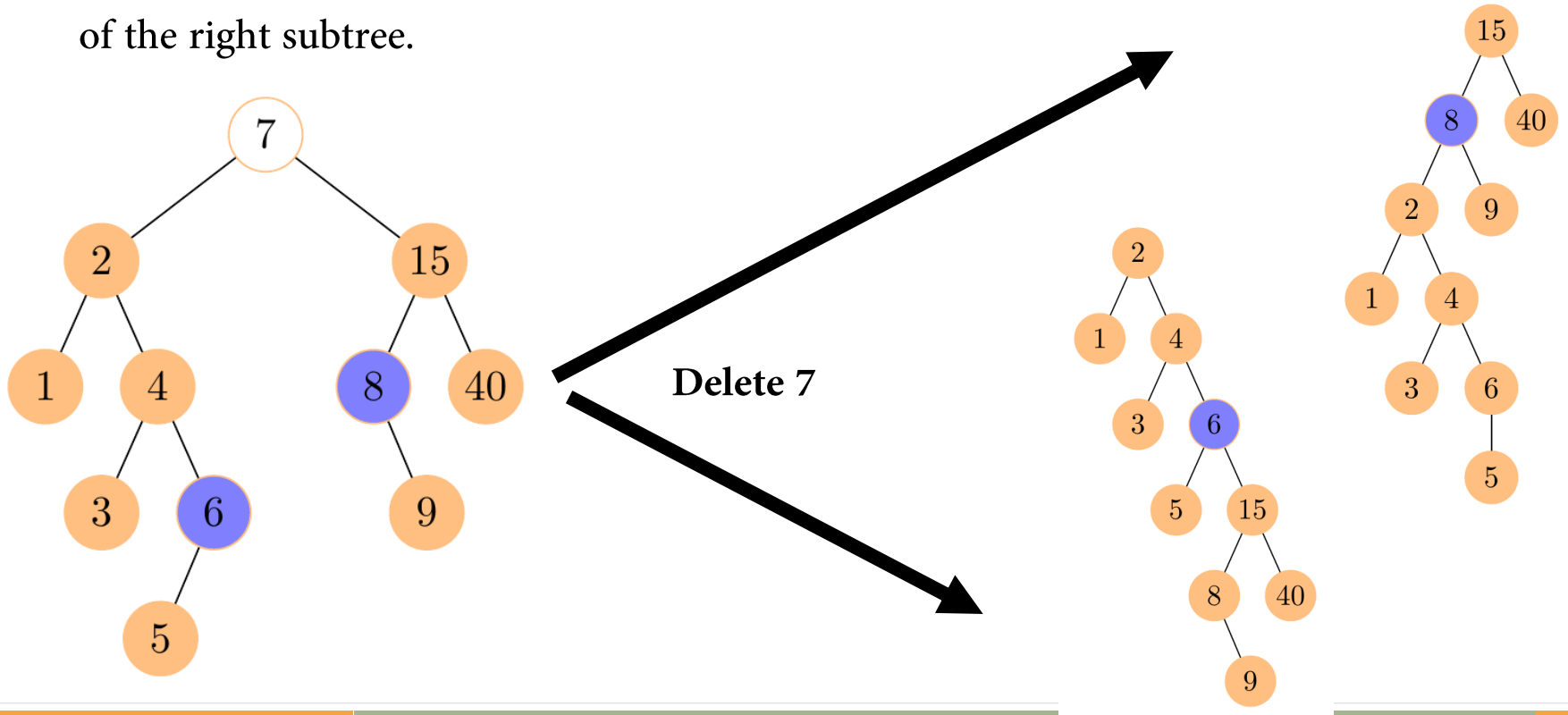
Deletion by Copying (Cont.)

- Example 2



Deletion by Merging…

- Idea: Remove the node and merge its left and right subtrees together while keeping the BST property
  - **Using the left subtree(in-order predecessor):** attach the right subtree to the rightmost node of the left subtree
  - **Using the right subtree(in-order successor):** attach the left subtree to the leftmost node of the right subtree.



Deletion by Merging…

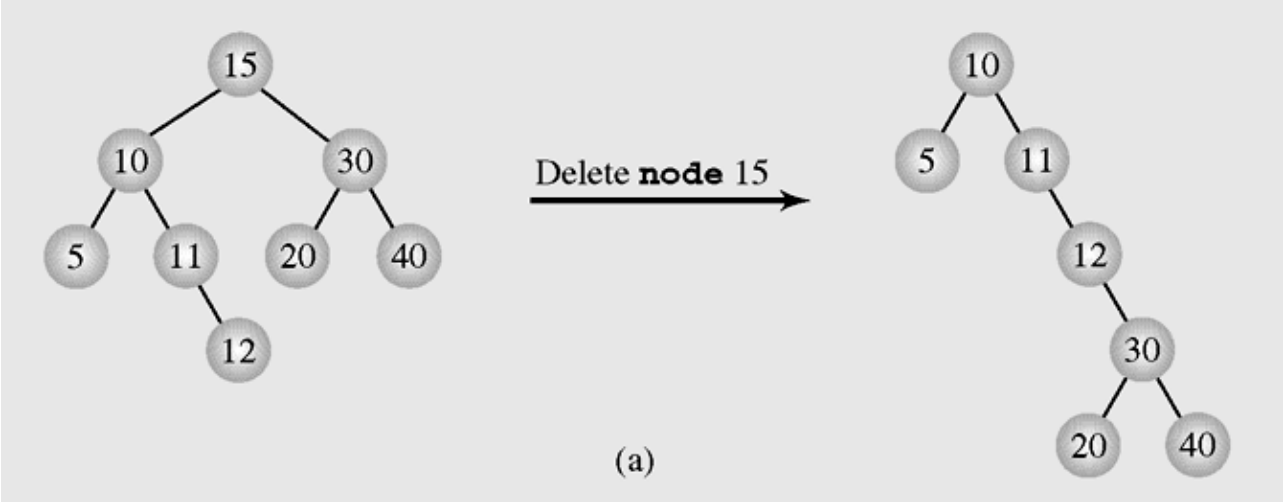
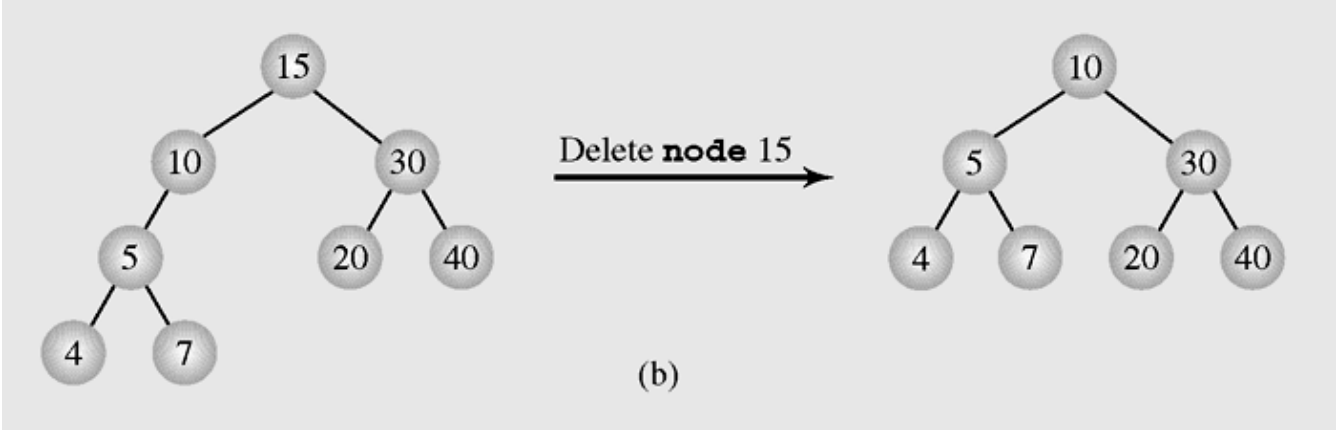


Figure 6-31 The height of a tree can be

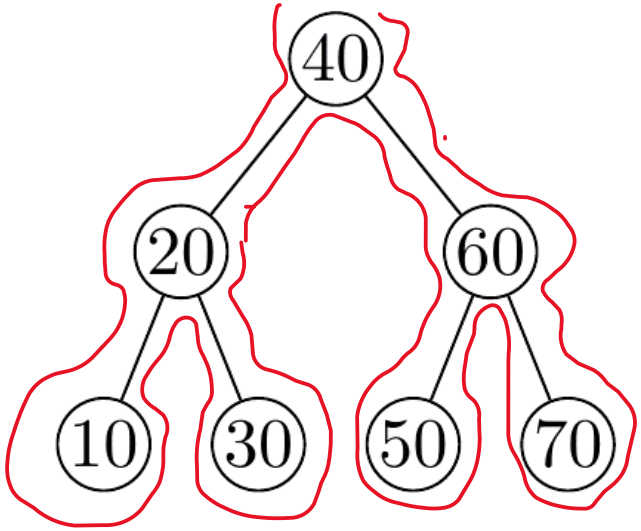
(a) extended or

(b) reduced after deleting by merging



BST Inorder Traversals

- The inorder traversal (L-N-R) of a BST produces an output that is sorted in increasing (ascending) order.
- The reverse inorder traversal (R-N-L) of a BST produces an output that is sorted in decreasing (descending) order.
- These properties are used in Tree-sort and in determining whether a given binary tree is a Binary search tree or not.



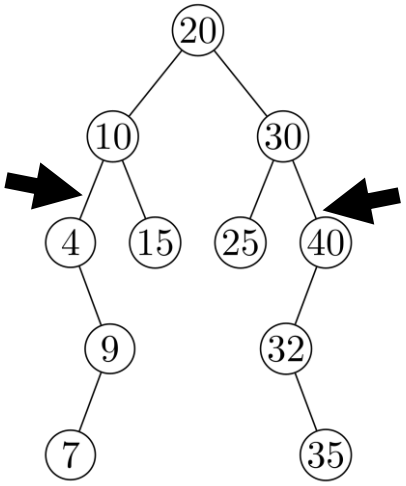
10, 20, 30, 40, 50, 60, 70

Binary Search Tree Implementation: getMin/getMax

- By the BST ordering property,
  - The minimum key is the key of the left-most node that has an empty left-subtree.
  - The maximum key is the key of the right-most node that has an empty right-subtree.

```
public T getMin(){
  if(isEmpty())
      return null;
  else
     return getMin(root);
}

private T getMin(BTNode node){
    if(node.left == null)
        return node.key;
    else
        return getMin(node.left);
}
```




```
public T getMin() {
    if (isEmpty())
        return null;

    BTNode current = root;

    while (current.left != null) {
        current = current.left;
    }

    return current.key;
}
```

- BSTs as Priority Queues:
  - By using insert and deleteMax or deleteMin a BST can be used as a priority queue in which the keys of the elements are distinct.

Binary Search Tree Implementation: size

- Each node contributes **1** to the total count.
- So, the total number of nodes is:
  - count = 1 + count of left subtree + count of right subtree

```
public int countNodes(){
      if(isEmpty())
         return 0;
      else
         return countNodes(root);
    }

public int countNodes(BTNode root) {
   if (root == null) {
       return 0;
   }
   return 1 + countNodes(root.left) + countNodes(root.right);
 }
```

Why BSTs?

- BSTs provide good logarithmic time performance in the best and average cases.
- Average case complexities of using linear data structures compared to BSTs:

Data Structure	Retrieval or Search	Insertion	Deletion	Traversal
BST	O(log n)	O(log n)	O(log n)	O(n)
Sorted Array	O(log n) (using Binary Search)	O(n)	O(n)	O(n)
Unsorted Array	O(n)	O(1)	O(n)	O(n)
Sorted Linked List	O(n)	O(n)	O(n)	O(n)
LinkedList	O(n)	O(1)	O(n)	O(n)

Note: BST worst execution time for each of the above operations is **O(n)** when the tree is linear (i.e., degenerate).

Why BSTs?...

- BSTs are used:
  - for indexing and multi-level indexing in databases.
  - to implement various searching algorithms.
  - in the implementation of sorting algorithms.
  - to implement dictionaries, dynamic sets, and lookup tables:
    - TreeMap and TreeSet data structures are internally implemented using self-balancing BSTs.
  - in Hash map implementations.
  - to represent memory to enable fast memory allocation.
  - for managing virtual memory areas in the Unix kernel.
  - to implement priority-queues.

Disadvantages of Binary Search Trees

- The overall shape of the tree depends on the order of insertion.
- If data is inserted in sorted order:
  - The tree becomes heavier(skewed) in one direction
  - The insert/search operations become expensive.

- The plain implementation of Binary Search Tree is not much used since it cannot guarantee logarithmic complexity.
  - Hence, we must use self-balancing implementations which are a bit more complex to implement.

The shape of a binary tree strongly affects the efficiency of its operations.